

ORIGINAL RESEARCH OPEN ACCESS

Enhancing Transmission Line Fault Classification and Prediction of Fault Location Using ML and DL Techniques

Subash Khanal¹  | Subarna Khadka¹  | Bipun Man Pati²  | Sudesh Parajuli¹ 

¹Department of Electrical and Electronics Engineering, Nepal Engineering College, Bhaktapur, Nepal | ²Artificial Intelligence Research Center, Advanced College of Engineering and Management, Kathmandu, Nepal

Correspondence: Subash Khanal (subashkhanal575@gmail.com)

Received: 17 May 2026 | **Revised:** 8 July 2026 | **Accepted:** 17 July 2026

ABSTRACT

Accurate fault classification and location in transmission lines plays a critical role in ensuring the reliability, stability and efficiency of the power system. Accurate fault detection and classification is key to safety, enabling brisk resolution of problems and mitigating electrical power supply disturbances. Traditional methods for fault type classification (FTC) and location prediction (LP) struggle to handle real-world fault situations, facing challenges like delays, limited capacity and vulnerability to single-point failures. This study develops and compares various machine learning (ML) and deep learning (DL) algorithms along with artificial neural network (ANN) to classify transmission line faults—LG, LLG, LL and three-phase faults—and predict fault locations. Fault data was generated using a MATLAB Simulink model incorporating phase voltages, currents, phase angles, zero-sequence, and negative-sequence components from both sending and receiving ends. The method achieved 99.69% and 99.78% accuracy using random forest and CatBoost for FTC, and R^2 of 1.00, MAE of 0.0080, and RMSE of 0.0729 for LP. Noise robustness was validated across SNR values (0–50 dB), achieving 90.53% FTC accuracy at 50 dB by CatBoost and R^2 of 0.9999 by random forest in LP. The study demonstrates exceptional performance of ML and DL models over traditional methods.

1 | Introduction

The reliability of electrical power systems has always been one of the central concerns of modern society. With an increasing dependence on electricity for both industrial and domestic purposes, even a minor interruption in supply can lead to widespread consequences. Among the many challenges facing power utilities, transmission line faults are the most critical. These faults may arise due to natural disturbances, such as lightning strikes, storms and falling trees, or due to equipment failure and human error. When a fault occurs, it causes abnormal current flow within the network, leading to instability, equipment damage, and sometimes complete system failure if not addressed quickly [1–5]. Different types of faults appear on transmission lines with varying probabilities. Line-to-ground (LG) faults are the most common, accounting for nearly three-fourths of all cases, whereas

line-to-line (LL), line-to-line- to-ground (LLG) and three-phase faults (LLL and LLLG) occur less frequently but often cause more severe disturbances [6, 7]. Regardless of type, the ability to promptly identify both the nature of the fault and its location is essential for ensuring safety, reducing downtime and minimizing economic loss.

Traditionally, utilities have relied on relay mechanisms and circuit breakers to detect and isolate faults [8–11]. Although effective to some extent, these methods are often slow and can be limited in accuracy, especially in large interconnected grids. Moreover, centralized detection systems suffer from communication delays, a single point of failure, and limited scalability [12–14]. As power networks evolve into complex, highly interconnected smart grids, these shortcomings have become more pronounced. In this context, the adoption of machine learning (ML) and deep

This is an open access article under the terms of the [Creative Commons Attribution-NonCommercial-NoDerivs](https://creativecommons.org/licenses/by-nc-nd/4.0/) License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.

© 2026 The Author(s). *IET Generation, Transmission & Distribution* published by John Wiley & Sons Ltd on behalf of The Institution of Engineering and Technology.

learning (DL) techniques provides a new dimension to fault detection and classification. Unlike conventional methods, The ML and DL models are able to learn from the patterns available in the electrical signals such as voltage and current phasors, sequence components such as zero sequence voltage (V0) and zero sequence current (I0), and symmetrical components. In this way, the models are able to adapt to changing conditions and offer accurate real-time fault detection and location [2, 15, 16]. By analysing voltage and current signals under various fault conditions, these algorithms can not only distinguish between fault types but also pinpoint their exact location with remarkable precision. This data-driven approach ensures quicker isolation of faulty sections and restoration of power, thereby enhancing grid reliability.

Over the last decade, research into ML and DL applications in power systems has shown encouraging results. Algorithms such as decision trees, random forests, and support vector machines (SVM) have consistently demonstrated high accuracy levels in simulated environments [17–20]. Moreover, their adaptability to large scale data makes them particularly suitable for smart grid applications [21–23], where vast amounts of information from sensors and monitoring devices are continuously generated. Therefore, the present study aims to build upon this growing body of knowledge by developing a comprehensive machine learning framework for fault type identification and location detection in transmission lines. The framework leverages simulation tools such as MATLAB Simulink to generate realistic datasets and applies multiple ML and DL algorithms to evaluate and compare performance. By doing so, it seeks to address the limitations of traditional techniques and contribute to more reliable, efficient and intelligent fault management in modern power systems.

As with the success of signal processing methods in detecting faults in the power system, the noise robustness of the machine learning models is an important consideration for their application in the real world environment in any field. As the methods involving wavelet feature extraction and signal to noise ratio analysis have been proven to be crucial for determining the faults in power equipment through analysing the noisy sensor data in transmission lines monitoring, understanding the response of machine learning models under varying noise conditions is equally important in developing robust algorithms [24–26]. In this paper, we use a systematic method similar to signal processing approaches for testing the robustness of the classification and regression algorithms against noise. Unlike our approach to signal processing in a particular field, we test a wide range of ten algorithms at six different signal to noise ratio (SNR) levels from 0 to 50 dB.

In this study, the voltage and current phasors of three-phase voltage and current at the sending and receiving ends of the faulted line, that is, VS1 and IS1 at the sending end and VR1 and IR1 at the receiving end of the faulted line, along with their phase angles and zero-sequence values of voltage and current V0 and I0, are used as input features for fault analysis of the transmission line system. In this paper, six types of faults are addressed, which are commonly encountered in power systems, that is, LG, LL, LLG, LLL, LLLG and no-fault condition. Two types of tasks are addressed in this paper: fault type classification using ten machine learning and deep learning algorithms such as random

forest classifier, XGBoost classifier, SVM classifier and MLP classifier, and fault location estimation using regression algorithms such as random forest regressor, decision tree regressor, gradient boosting regressor and artificial neural network regressor.

1.1 | Contributions with Previous Related Work

The proposed work has been inspired by several earlier works that used ML/DL for the classification and location of faults in transmission lines, but with some important caveats that this work addresses. Other studies, like [1, 17] were only concerned with fault classification and not estimation of fault location at the same time. Previous research on location prediction problems [8, 27, 28] has neglected to account for the zero and negative sequence components of the problem, or used a restricted set of fault types (usually LG and LLL) [29–31]. These have been important in the ability to distinguish grounded from ungrounded three-phase faults. Also, noise robustness testing over a systematic range of SNR levels (0–50 dB) has not been used together with a dual task paradigm. While both [32–34] and [35–37] used single-model architectures, this study makes a comprehensive comparison of 10 classifiers and 9 regressors, all under identical conditions, and offers a reproducible comparison. Developed for FTC and LP on the same 66 km, 132 kV simulated line, with the zero-sequence characteristics and multi-level noise injection, this is a major distinguishing contribution of this work.

1.2 | Objectives

Although there are several works on individual fault classification or location prediction tasks, no comprehensive works that simultaneously tackle both tasks with rigorous algorithm comparison, feature engineering of zero sequence and systematic evaluation of the noise robustness of the algorithms have been thoroughly investigated. The majority of current approaches either (a) treat FTC and LP as distinct problems and train using different sets of data, (b) do not assess the performance of the model in realistic noisy settings over a broad range of SNRs or (c) employ only a few algorithms, restricting the generalizability of conclusions. In this study all three gaps are tackled in one integrated approach.

2 | Methodology

The methodology for this study, outlined in Figure 1, encompasses data generation, feature engineering, model development and evaluation for two distinct tasks: FTC and LP. A model for a transmission line was implemented in MATLAB Simulink to simulate five distinct fault types and generate corresponding voltage and current data. Fault data were captured at the moment of fault inception, while non-fault data were sampled during normal operating conditions. The complete dataset was then partitioned into training and testing subsets with an 80:20 ratio. Min-max normalization was applied to pre-process the data. For FTC, we trained and evaluated nine machine learning algorithms—decision tree, random forest, SVM, KNN, XGBoost, naive Bayes, AdaBoost, gradient boosting, and CatBoost—along an artificial neural network (ANN). For location prediction (LP), a convolutional neural network (CNN) was utilized. The

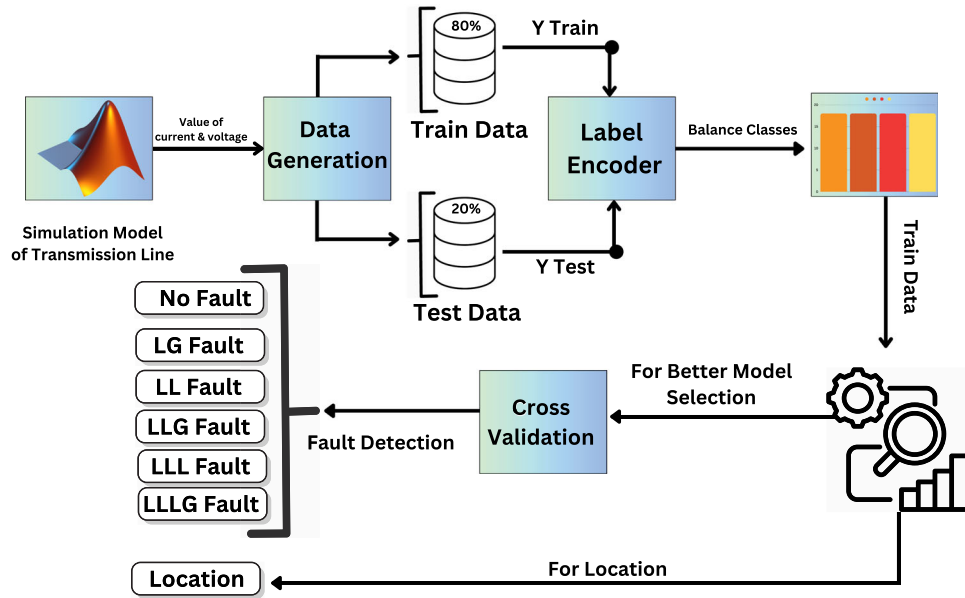


FIGURE 1 | Framework to classify and locate fault.

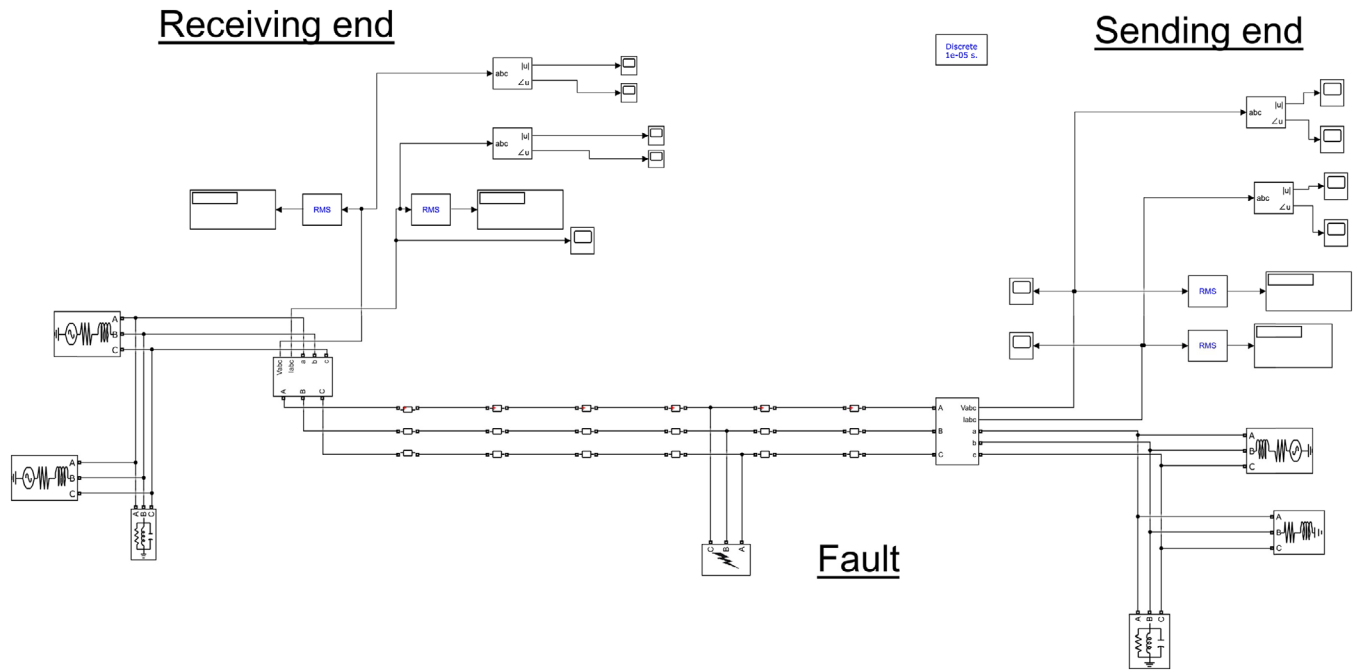


FIGURE 2 | Transmission line model for data generation.

optimal model for each task was selected based on the highest classification and prediction accuracy.

2.1 | Transmission Line Model and Data Generation

The MATLAB Simulink model used for data generation is depicted in Figure 2. The model comprises key components of a transmission line system. A three-phase source block acts as generation source and PQ bus. Voltage and current measurements at the sending and receiving ends are taken using a three-phase

VI measurement block, whose outputs are converted to RMS values by an RMS block. The transmission line is modelled as a 66 km, 132 KV (line-to-line) using three-phase distributed parameters block. A three-phase load block represents the load. For visualization, a display block shows the per-phase voltages and currents, while a scope captures the waveforms for analysis and facilitates data logging. A sequence analyser block is incorporated to calculate the positive, negative and zero-sequence components, which are crucial for accurate fault location prediction. To create a robust dataset covering diverse fault scenarios, simulations were conducted with systematic variations in three key parameters:

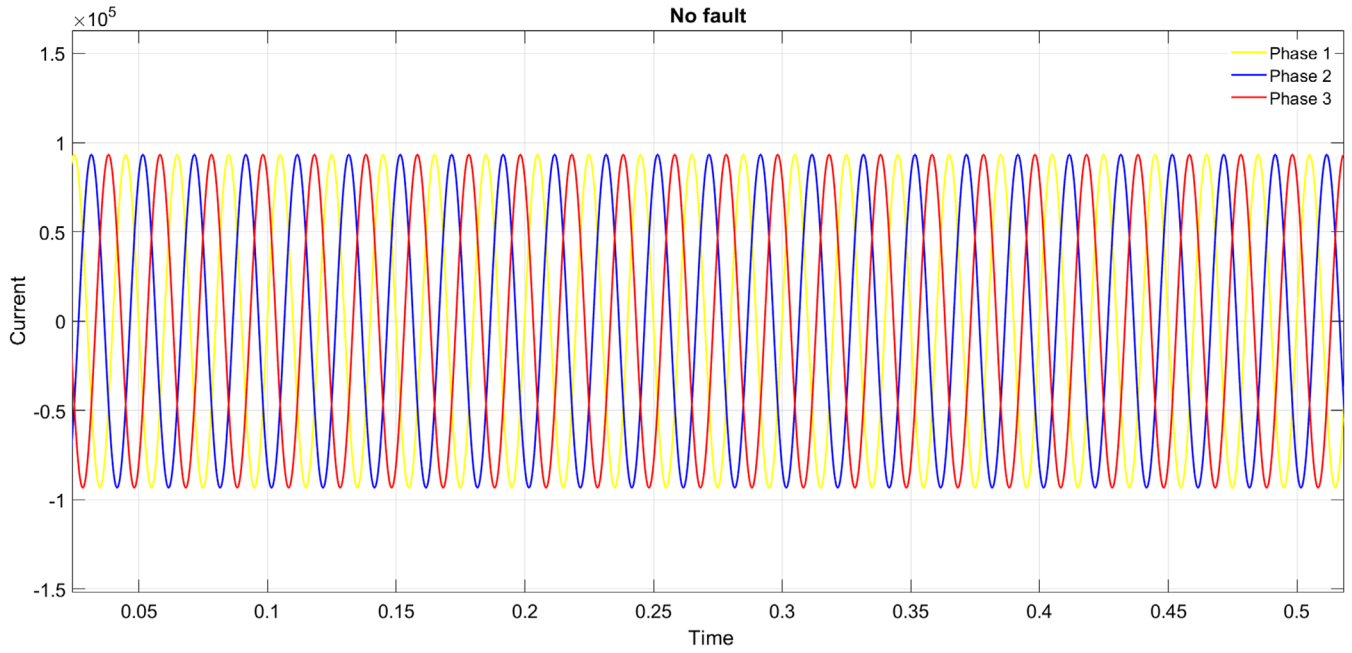


FIGURE 3 | No fault current graph.

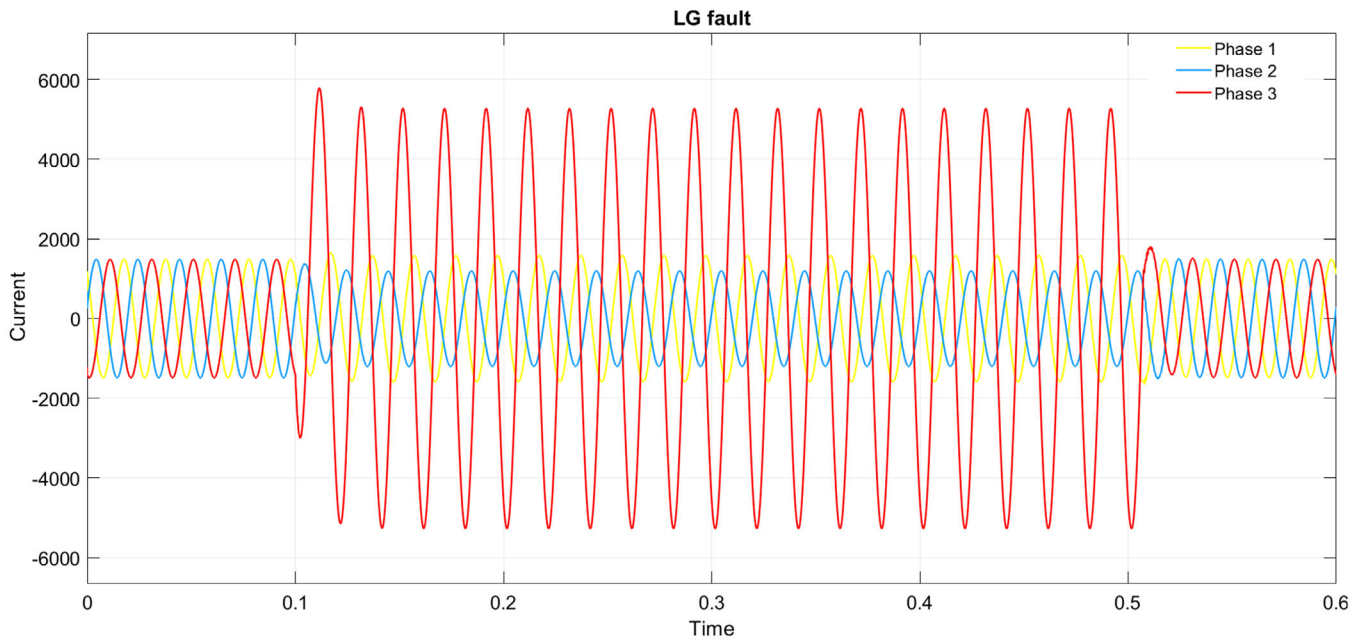


FIGURE 4 | LG fault current graph.

1. Fault type: All ten possible shunt faults were simulated: LG (three subtypes), LL (three subtypes), LLG (three subtypes) and LLL.
2. Fault location: Faults were applied at incremental distances of 1 km along the line (from 1 to 65 km), resulting in 65 distinct location points per fault type.
3. Fault inception angle: To account for transients, faults were initiated at angles varying from 0° to 315° in steps of 45° relative to the phase-A voltage zero-crossing.
4. Fault resistance: A range of impedances (0.001, 0.1, 1, 10, 50Ω) was used to simulate both solid and high-impedance faults.

The output current graphs of no fault, LG, LL, LLG, LLL and LLLG faults are shown in Figures 3, 4, 5, 6, 7 and 8, respectively. In case of No Fault all the phase currents are normal without any change in the graph whereas, when the fault has occurred the faulty phase current has significantly increased demonstrating that the fault has occurred in the particular phase.

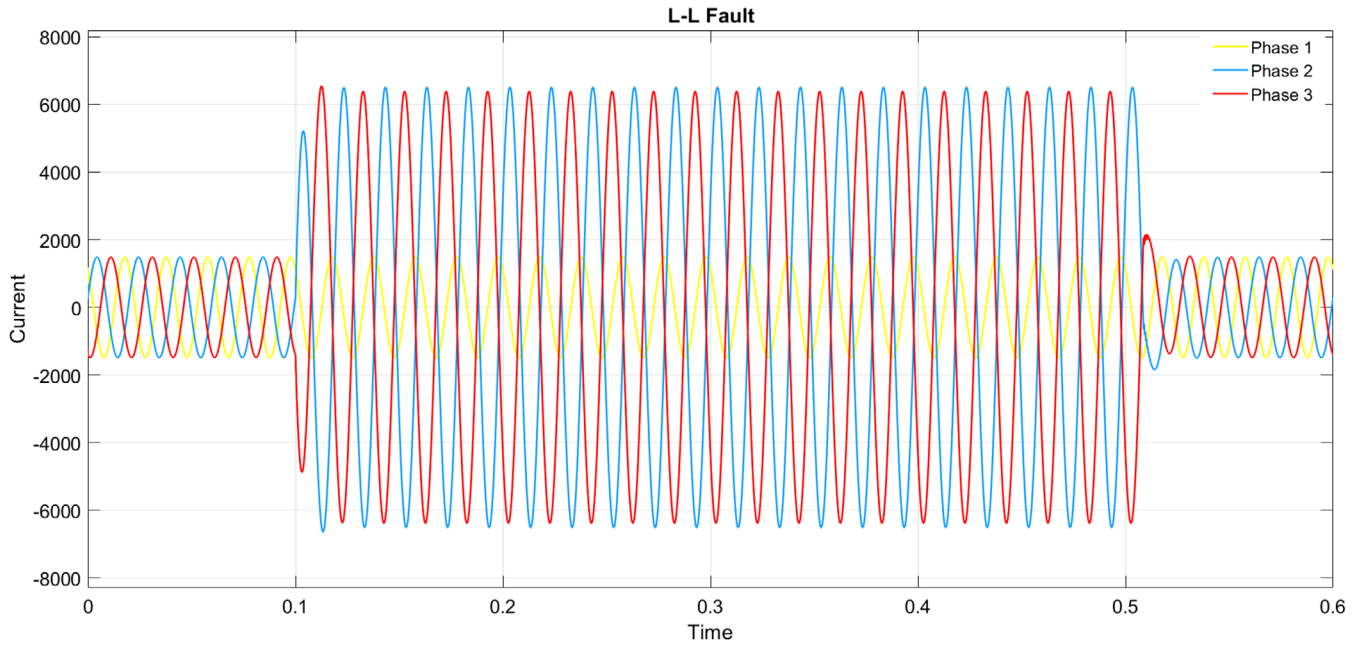


FIGURE 5 | LL fault current graph.

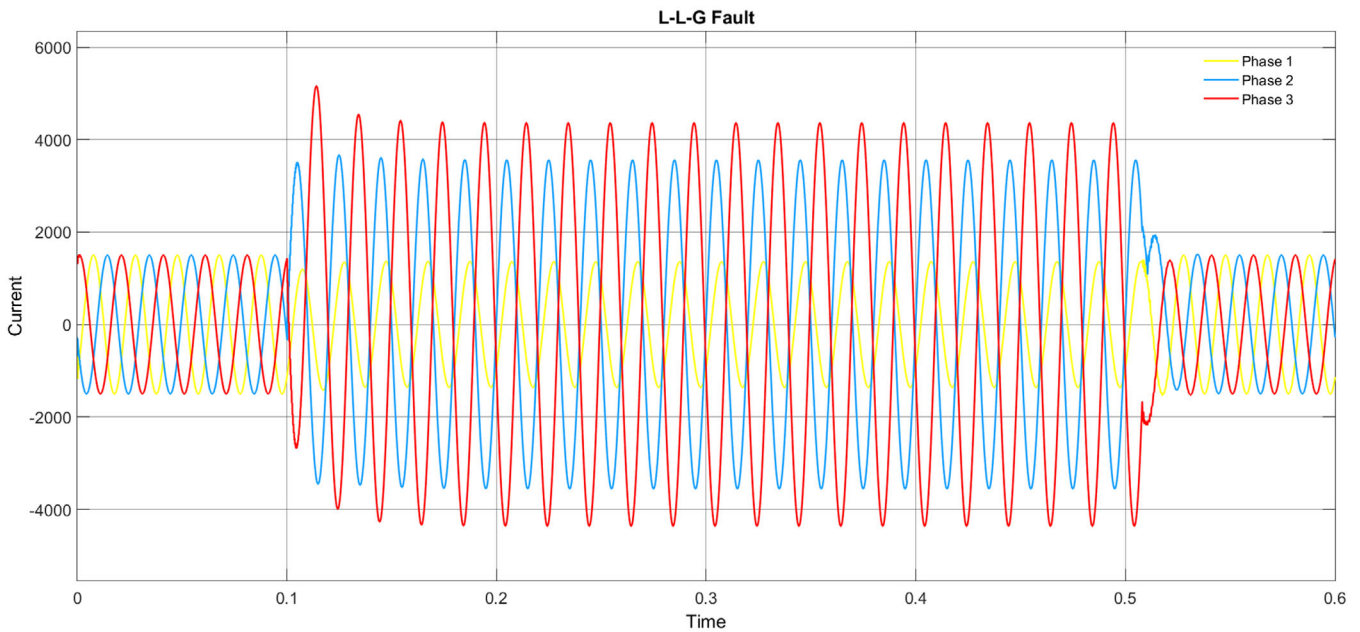


FIGURE 6 | LLG fault current graph.

2.2 | Feature Extraction and Preprocessing

The feature extraction process has been implemented in a structured signal processing pipeline in the MATLAB/Simulink environment. The three-phase VI measurement blocks at both sending and receiving ends record the voltage and current waveforms at the moment of fault inception. The RMS blocks then convert these waveforms into root mean square values to obtain the per-phase magnitudes V_A , V_B , V_C , I_A , I_B and I_C . The sequence analyser block performs the symmetrical component transformation (Fortescue transform) to decompose the three-

phase phasors into their positive-sequence (V_1 , I_1), negative-sequence (V_2 , I_2), and zero-sequence (V_0 , I_0) components, as given in Equations (1–3):

$$V_0 = \frac{1}{3}(V_a + V_b + V_c) \quad (1)$$

$$V_1 = \frac{1}{3}(V_a + aV_b + a^2V_c) \quad (2)$$

$$V_2 = \frac{1}{3}(V_a + a^2V_b + aV_c) \quad (3)$$

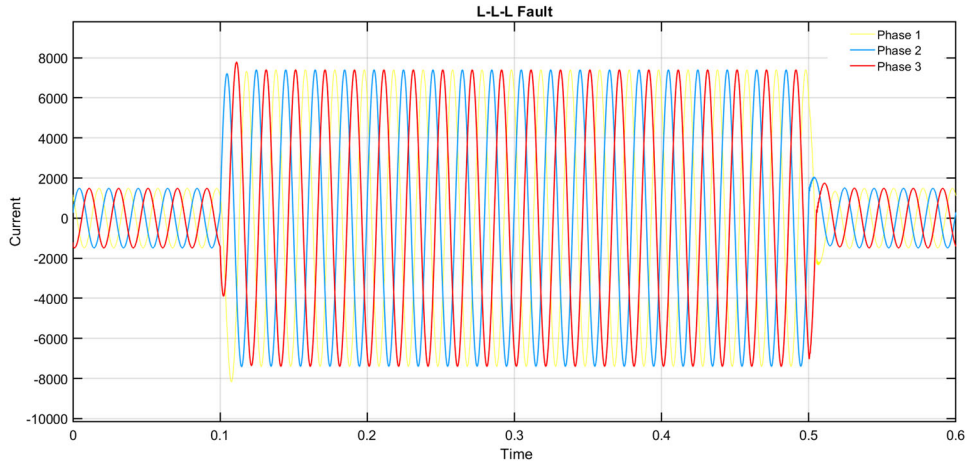


FIGURE 7 | LLL fault current graph.

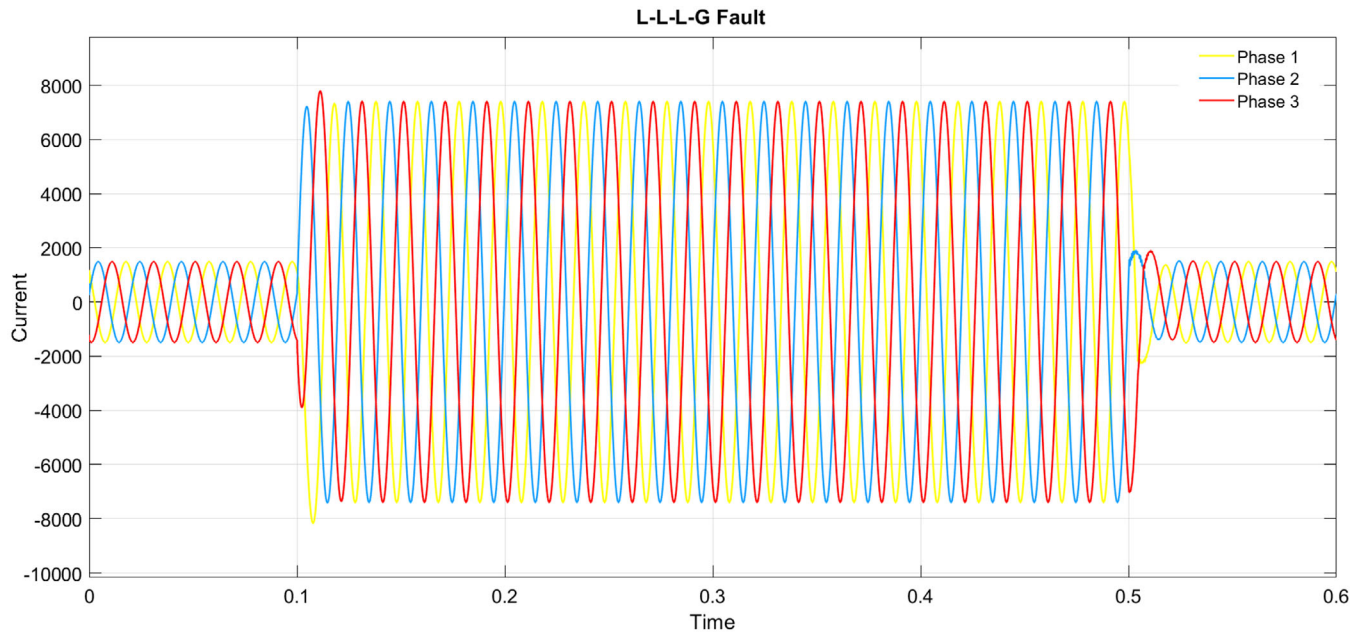


FIGURE 8 | LLLG fault current graph.

where $a = e^{j2\pi/3}$. The magnitude and phase angle of the positive-sequence voltage and current at both ends — V_{S1} , $\angle V_{S1}$, V_{R1} , $\angle V_{R1}$, I_{S1} , $\angle I_{S1}$, I_{R1} , and $\angle I_{R1}$ —are extracted for use in fault location regression. For fault type classification, the features V_A , V_B , V_C , I_A , I_B , I_C , V_0 and I_0 are recorded at the moment of fault inception. All feature values are stored in a data file at every simulation step, producing a structured tabular dataset. No hand-crafted time-frequency features (e.g., wavelet coefficients) are used; the features are directly derived from steady-state phasor quantities measured at both ends of the line.

Fault type classification (FTC): Inputs are taken from the simulated electrical measurements of the transmission line model and include the three-phase voltage components (V_A , V_B , V_C), three-phase current components (I_A , I_B , I_C) and zero-sequence voltage (V_0) and current (I_0) as a result of symmetrical component transformation. RMS values only are used for these eight features,

as the type of fault is best determined by which phases are involved in the fault, and the relative magnitude of the various phase quantities and zero sequence components, rather than the absolute phase angles. The zero-sequence components are provided for all fault types (LG, LL, LLG, LLL, LLLG, no fault), but are more important for asymmetrical faults (LG and LL) where currents take unbalanced paths and hence give non-zero V_0 and I_0 ; while symmetrical faults (LLL and LLLG) give an approximately zero value for the zero-sequence component.

In contrast to FTC, in fault location prediction the magnitudes of the voltage and current at both the sending end (V_{S1} , V_{R1}) and receiving end (I_{S1} , I_{R1}) are used, along with their respective phase angles ($\angle V_{S1}$, $\angle V_{R1}$, $\angle I_{S1}$, $\angle I_{R1}$), which encode the spatial information required for accurate distance estimation calculation and are inputted to the ML/DL regression model to estimate the fault distance in kilometres.

TABLE 1 | Example of training and testing dataset (5 LG and 5 LLG samples).

Time	IA	IB	IC	VA	VB	VC	Fault	V0	I0
0.19987	4666.74	539.08	-1082.30	21,676.55	-78,166.56	83,419.57	LG	14,859.97	1410.10
0.19988	4659.01	543.75	-1083.89	21,872.70	-78,316.84	83,268.07	LG	14,862.65	1410.89
0.19989	4651.22	548.42	-1085.47	22,068.63	-78,466.34	83,115.74	LG	14,865.28	1411.67
0.19990	4643.39	553.08	-1087.04	22,264.35	-78,615.07	82,962.60	LG	14,867.86	1412.45
0.19991	4635.52	557.74	-1088.59	22,459.85	-78,763.02	82,808.63	LG	14,870.41	1413.23
0.20000	4562.59	599.41	-1102.12	24,209.11	-80,059.43	81,386.36	LLG	14,891.38	1420.21
0.20001	4554.26	604.01	-1103.57	24,402.30	-80,199.55	81,224.29	LLG	14,893.50	1420.98
0.20002	4545.89	608.60	-1105.01	24,595.26	-80,338.87	81,061.42	LLG	14,895.57	1421.74
0.20003	4537.47	613.19	-1106.44	24,787.97	-80,477.40	80,897.75	LLG	14,897.59	1422.51
0.20004	4529.01	617.77	-1107.85	24,980.43	-80,615.13	80,733.28	LLG	14,899.58	1423.28

The data set used for this study was synthetically created by utilizing MATLAB/Simulink programming software, in which a model of a transmission line was simulated for various fault conditions and creates labelled data for the electrical parameters. As the data was created programmatically for various fault conditions, the data is naturally clean and free from noise and other issues such as missing values. Since, the data was created for all types of faults such as LG, LL, LLG, LLL, LLLG and no fault equally, there was not need of using any oversampling or undersampling techniques such as SMOTE or class weighting. Even though the data is naturally clean, various preprocessing steps were followed to make the data numerically consistent. The steps are:

1. Missing value imputation

The missing values in the dataset were addressed using mean imputation, where each missing value was replaced by the arithmetic mean of its corresponding feature column. This is represented by Equation (4).

$$\text{Imputed Value} = \frac{1}{n} \sum_{i=1}^n x_i \quad (4)$$

where, x_i are non-missing values, and n is the count of non-missing values.

2. Label encoding

The categorical target variable fault type, with class labels (LG, LL, LLG, LLL, LLLG, no fault), was mapped to numerical integer values by label encoding which is shown by Equation (5).

$$y_{\text{encoded}} = \mathcal{L}(y_{\text{label}}) \quad (5)$$

where, y_{label} is the original categorical fault type label, y_{encoded} is the corresponding integer class index, and $\mathcal{L}$ denotes the label encoding mapping function.

3. Z-score standardization

To ensure that all input features contribute equally to the model and no feature dominates the model based on the magnitude of its values, Z-score standardization was employed by StandardScaler: The formula for Z-score standardization is shown by Equation (6).

$$z = \frac{x - \mu}{\sigma} \quad (6)$$

where, z is the standardized feature value, x is the original feature value, μ is the mean of the feature computed from the training data and σ is the standard deviation of the feature computed from the training data.

2.3 | Fault Type Classification

Once the datasets were preprocessed and balanced, the machine learning classifiers were trained. 60,000 data were used to train the classifiers. The data was divided in the 80% training set and 20% were used as test data considering the number of data used. Label encoder was used to divide the data into equal parts so the model gets trained well. The sample of dataset is shown in Table 1 transmission line where phase currents IA, IB and IC are in Ampere, phase voltages VA, VB, VC are in volt and time is in seconds. A total of nine classifiers, that is, decision tree, gradient boosting, random forest, SVM, K-nearest neighbours (KNN), XGBoost, naive Bayes, AdaBoost and CatBoost. were used for training based on their widespread application in classification problems and proven performance in electrical fault detection research. Along with all the traditional machine learning classifiers, a neural network model was also developed to enhance the accuracy of fault classification. The histogram in Figure 9 shows the distribution of input data, that is, three phase voltages (VA, VB, VC) and three phase currents (IA, IB, IC). Similarly, Figure 10 shows the distribution of data when the sequential components were added to improve the accuracy of model. To evaluate the fault classification model's performance, various metrics such as accuracy, F1-score, precision and recall are used. These are derived from the confusion matrix, which

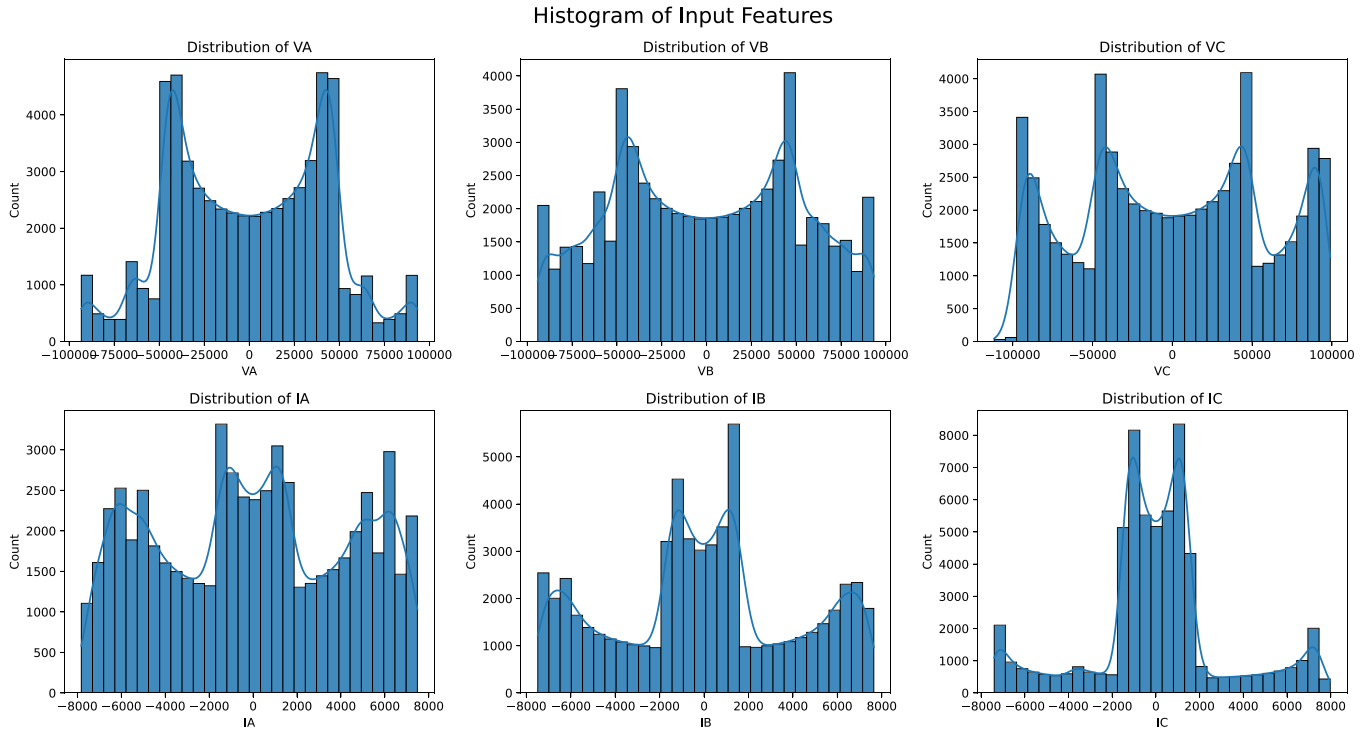


FIGURE 9 | Histogram for input data visualization.

compares actual and predicted fault types. The evaluation of accuracy is given by Equation (7).

$$\text{Accuracy} = \frac{(TP + TN)}{(TP + TN + FP + FN)} \times 100\%, \quad (7)$$

where, TP, TN, FP and FN refers to actually faulty, actually not faulty, not faulty but predicted as faulty, and faulty but predicted as not faulty, respectively. Precision is given by Equation (8) which indicates the proportion of correctly predicted positive cases out of all predicted positives.

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (8)$$

The recall is given by Equation (9) which measures the ability of the model to correctly identify actual positive cases.

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (9)$$

F1-score is given by Equation (10) which uses the harmonic mean of precision and recall, giving a balanced measure of both.

$$\text{F1-score} = \frac{2 \times P \times R}{P + R}. \quad (10)$$

2.4 | Fault Location Prediction

Regression models were used to predict the location of the fault accurately. Five different ML algorithms, that is, XGB regressor, linear regressor, random forest regressor, decision tree regressor, and gradient boosting regressors were used. CNN was used as a DL regressor for predicting the fault location. To train the models, a methodical approach is employed based on key electrical

parameters and their real-time measurements. The significance of Figure 11 is to illustrate the diversity and distribution of the features extracted, which are used to predict fault locations. The voltage, current magnitudes, phases and distance from the fault covered a wide range, which indicates that the data set represents various fault conditions (during different fault distances, fault resistance and fault inception angle). The following feature diversity allows ML/DL regression models to learn the nonlinear relationships between electrical parameters and fault distance. The combination of these parameters was used within a well-established fault location. The distributions of the input features represented by the Figure 11 are statistical distributions of the positive-sequence voltage magnitudes (VS1, VR1), the positive-sequence current magnitudes (IS1, IR1), phase angles of the positive-sequence currents (alpha S, alpha R), phase angles of the positive-sequence voltages (beta S, beta R), and the fault distance (r12). As it can be noticed in the histograms, the feature distributions are wide and non-uniform, which indicates the diversity of fault conditions simulated (location, type, resistance and inception angle). The distribution of VS1 and IS1 values show that the dataset covers the full spectrum of system operating conditions in fault. The Distance variable (target) exhibits a fairly uniform distribution between 1 and 65 km, which reflects a good spatial coverage across the transmission line. This diversity of feature distributions enables the use of nonlinear ML/DL regression models, which would be inadequate if one used only a linear model. The classical two-ended impedance-based fault location formula for the transmission line is implemented in Equation (11), which is based on the positive-sequence equivalent network model of the transmission line. The per-unit distance m to the fault from the sending end is calculated from the ratio of the positive-sequence voltages at both ends ($V1s/V1r$) and the positive-sequence current at the receiving end ($I1r/(I1s + I1r)$),

Histogram of Input Features with Sequence

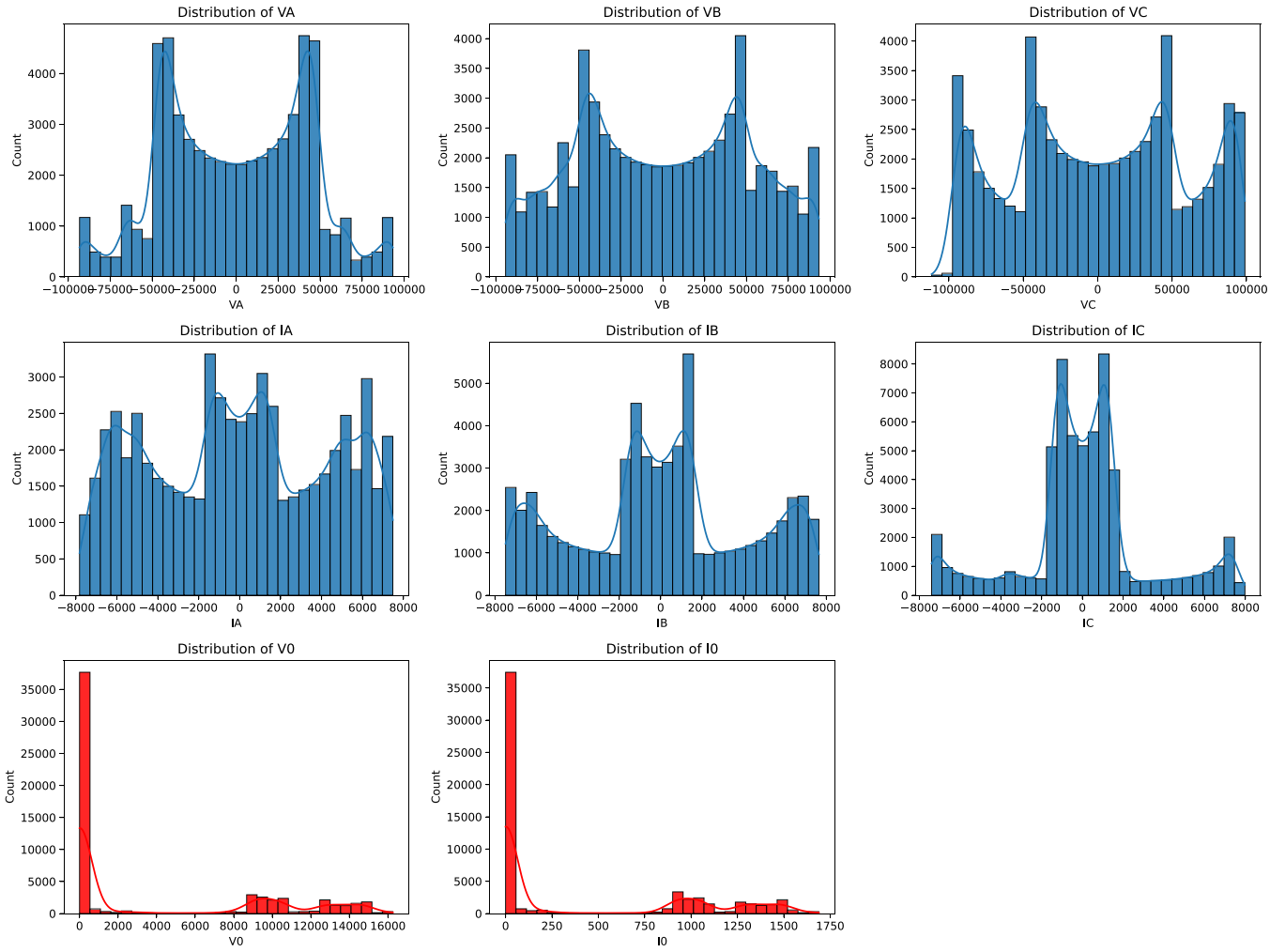


FIGURE 10 | Histogram for input data visualization after addition of sequence components.

with the total line impedance Z_L . This formula is only valid in a balanced network and is a distance estimation without regard to fault resistance for metallic faults. The per-unit distance m is then multiplied by the physical line length (66 km in this study) in Equation (12) to get a real distance in kilometres. Equation (11) defines the target variable m , which is the estimated fault distance from the sending end of the transmission line. It is the dependent output variable for the fault location regression models used in this study.

$$m = \frac{V_{1s} - V_{1r} + (I_{1r} \times Z_L)}{(I_{1s} \times Z_L)}, \quad (11)$$

where, m , V_{1s} , V_{1r} , I_{1r} , Z_L refers to per-unit distance to fault, sending end positive sequence voltage, receiving end positive sequence voltage, receiving end positive sequence current and transmission line impedance, respectively. I_1 is the sum of I_{1s} and I_{1r} . The actual fault location can be calculated according to Equation (12).

$$\text{Actual} = m \times \text{actual length of line}. \quad (12)$$

The derivation of Equation (11) follows from writing two independent loop voltage equations for the fault-point voltage V_f , one

from each terminal. From terminal S:

$$V_f = V_{1s} - I_{1s} \cdot m \cdot Z_L.$$

From terminal R:

$$V_f = V_{1r} + I_{1r} \cdot (1 - m) \cdot Z_L.$$

Since both expressions represent the same fault-point voltage, setting them equal and solving for m eliminates V_f algebraically, yielding Equation (11). This is the key advantage of the two-ended approach over single-ended fault location methods: because V_f is solved for directly rather than estimated from local measurements alone, the fault resistance R_f does not appear in the final expression for m , making the calculation inherently insensitive to fault resistance for metallic faults in a balanced network [8].

The evaluation of regression models was done by using three parameters, which were mean absolute error (MAE), root mean squared error (RMSE), and coefficient of determination (R^2). MAE measures the average of the absolute differences between predicted and actual values as shown in Equation (13).

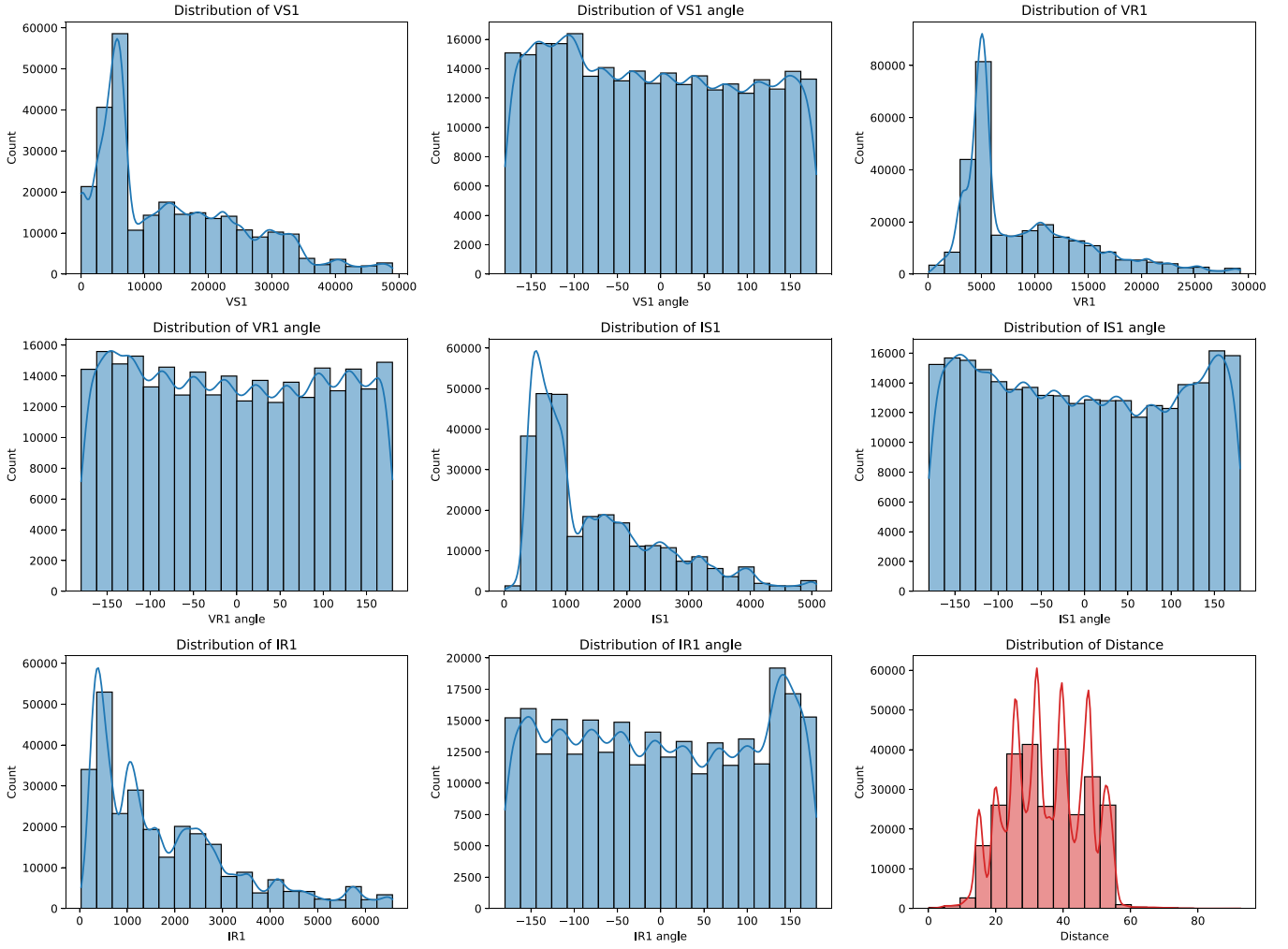


FIGURE 11 | Histogram for data visualization of regression.

It measures the average magnitude of errors in predictions. Lower MAE indicates higher accuracy.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (13)$$

where, y_i , $\hat{y}_i$ and n refers actual value, predicted value and number of samples respectively. RMSE gives the square root of the average of squared errors, penalizing larger errors more as shown in Equation (14). It reflects the square root of the average squared differences between predicted and actual fault locations. It penalizes larger errors more than MAE.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad (14)$$

R^2 shown in Equation (15) explains how well the predicted values approximate the actual data. It indicates how well the model explains the variability of the fault location. R^2 value close to 1.0 signifies excellent performance.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad (15)$$

where, $\bar{y}$ is the mean of actual values.

2.5 | ROC Curve

The ROC curve evaluates classifier performance across all decision thresholds by plotting the true positive rate (TPR) against the false positive rate (FPR), defined as:

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN} \quad (16)$$

The area under the ROC curve (AUC) summarizes discriminative ability as:

$$AUC = \int_0^1 TPR(FPR) d(FPR) \quad (17)$$

where TP, FP, FN and TN denote true positives, false positives, false negatives, and true negatives respectively. An AUC of 1.0 indicates perfect classification while an AUC of 0.5 indicates random chance performance.

2.6 | Neural Network Architecture

The model of artificial neural network (ANN) was created for multi-class fault classification. It has an input layer of 8 neurons

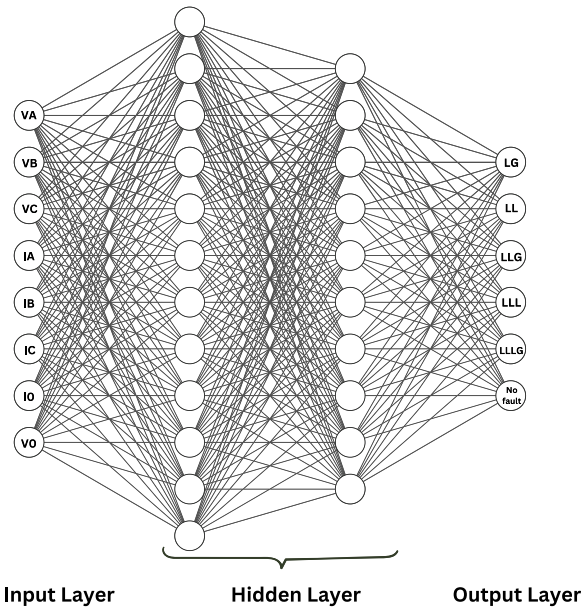


FIGURE 12 | ANN architecture.

corresponding to the chosen features, two hidden layers of 100 and 50 neurons respectively, where the ReLU activation function is used to introduce non-linearity into the model. Finally, there is an output layer of 6 neurons corresponding to the six types of faults, where the Softmax activation function is used. The architecture of ANN can be visualized by Figure 12. The fault location prediction pipeline, which uses a separate regression model with positive-sequence phasor features (V_{S1} , V_{R1} , I_{S1} , I_{R1}) and their phase angles as inputs, is described in detail in Section 2.4.

2.7 | Hyperparameter Tuning Strategy

Hyperparameter optimization was carried out using a library called “RandomizedSearchCV” for two models: “random forest” and “CatBoost.” This was chosen over “grid search” for optimization because of the computational efficiency it provides over “grid search,” whereby a fixed number of combinations of hyperparameters are randomly sampled from a given search space instead of evaluating all possible combinations of parameters, especially for models with a large number of hyperparameters.

The search space for the “random forest” model was set as follows: the number of estimators, which corresponds to the total number of decision trees in the forest, was set to search over 50, 100; the maximum depth of the individual trees in the forest, which prevents overfitting of the trees by limiting how deep they are allowed to grow, was set to search over None, 10, 20; the minimum number of samples required to split an internal node was set to search over 2, 5; the minimum number of samples required at each leaf node was set to search over 1, 2; and finally, bootstrap was set to true. In the case of the CatBoost model, the search space for the hyperparameters was defined as: the number of boosting iterations, which determines the total number of trees constructed sequentially, was set to 200, 500, 800; tree depth, which determines the complexity of each tree constructed, was set to 4, 6, 8, 10; the learning rate, which determines the contribution

of each tree to the final prediction, was set to 0.01, 0.03, 0.05, 0.1; the coefficient of the L2 regularization penalty term, which determines the complexity of the model to prevent overfitting, was set to 1, 3, 5, 7, 9; and border count, which determines the number of splits for numerical features, was set to 32, 64, 128.

A 5-fold cross-validation approach was adopted for the tuning of the models within the training set. In each of the five folds of the cross-validation approach, the training set was again split such that 80% of it was used for the fitting of the models and the rest 20% was used for the validation set. The combination of the hyperparameters that resulted in the highest mean accuracy for the validation set was chosen as the optimal combination of the hyperparameters for the respective models. The models were then re-trained on the entire training set using the optimal set of hyperparameters and then tested on the test set for the purpose of obtaining the performance metrics of the models.

2.8 | Noise Injection

In order to provide realistic operational settings, additive white Gaussian noise (AWGN) is introduced to the voltage and current signals. AWGN is very common because of its capacity to represent thermal and measurement noises in the power systems environment.

The noisy signal is shown in Equation (18).

$$x_{\text{noisy}}(t) = x(t) + n(t), \quad (18)$$

where, $x(t)$ represents the original clean signal, $n(t)$ denotes the noise component and $x_{\text{noisy}}(t)$ is the resulting corrupted signal.

The noise is assumed to follow a Gaussian distribution which is expressed in Equation (19).

$$n(t) \sim \mathcal{N}(0, \sigma^2), \quad (19)$$

where, σ^2 represents the variance of the noise and the mean is zero.

The level of noise is controlled using the signal-to-noise ratio (SNR), which is shown in Equation (20).

$$\text{SNR (dB)} = 10 \log_{10} \left(\frac{P_{\text{signal}}}{P_{\text{noise}}} \right), \quad (20)$$

where, P_{signal} is the average power of the original signal and P_{noise} is the noise power.

The signal power is calculated using Equation (21).

$$P_{\text{signal}} = \frac{1}{N} \sum_{i=1}^N x_i^2, \quad (21)$$

where, N is the total number of samples.

Since, the noise power is equal to the variance of the Gaussian noise as shown in Equation (22).

$$P_{\text{noise}} = \sigma^2. \quad (22)$$

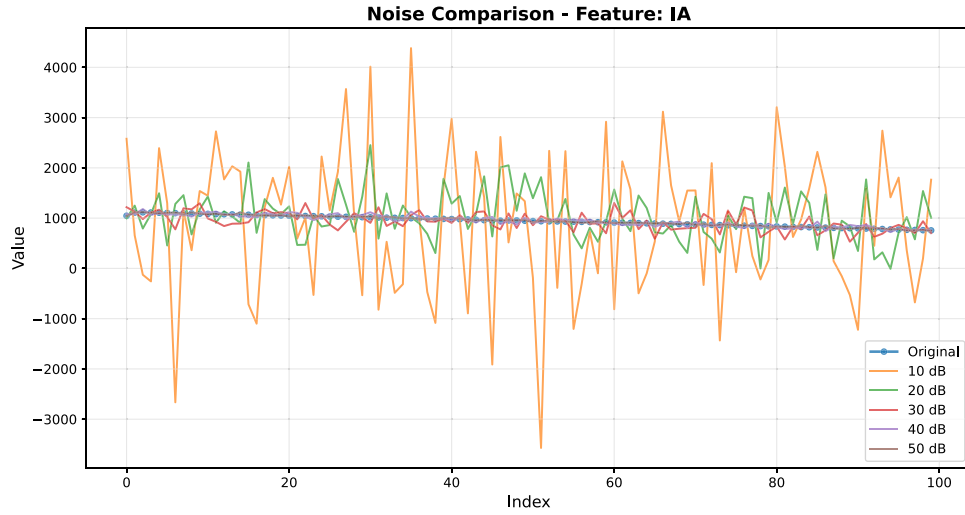


FIGURE 13 | Comparison of noise in SNR and original signal.

The required noise standard deviation for a given SNR level is computed as shown in Equation (23).

$$\sigma = \sqrt{\frac{P_{\text{signal}}}{10^{\text{SNR}/10}}} \quad (23)$$

In this study, six SNR levels are considered: 0 (noise-free baseline), 10, 20, 30, 40 and 50 dB. Lower SNR values correspond to higher noise levels, making the prediction task more challenging. Noise is independently added to each feature and sample to ensure statistical consistency and avoid bias in the dataset. The comparison noise in SNR and output signal can be visualized in Figure 13.

3 | Result and Analysis

3.1 | Fault Classification

3.1.1 | Test Set Performance Before Feature Engineering

Table 2 shows the test set performance of all the ten models on the held-out 20% test data set prior to feature engineering, measured using precision, recall, F1-score and accuracy. In the initial setup, only three-phase voltages (VA, VB, VC) and three-phase currents (IA, IB, IC) were considered as inputs without any other engineered features. Among the ten machine learning models, random forest delivered the highest accuracy of 93.20%, followed by decision tree at 92.45%, CatBoost at 86.58%, and XGBoost at 86.40%. Similarly, the performances of KNN, gradient boosting and neural Networks were relatively similar with accuracies of 85.12%, 84.85% and 84.80% respectively. SVM model achieved an accuracy of 81.18%, whereas naive Bayes and AdaBoost had the lowest accuracy of 58.33% and 48.57% respectively, indicating that the two algorithms were not suitable for FTC.

3.1.2 | Test Set After Feature Engineering

After the initial evaluation, feature engineering was carried out using zero sequence voltage (V0) and zero sequence current (I0)

as new features along with the original three phase voltage and current readings. The zero sequence components appear only during asymmetrical faults like LG and LLG but are negligible in case of symmetrical three phase faults like LLL and LLLG. The discriminative nature of this process helped in distinguishing between different types of faults based on their three phase voltage and current magnitudes, especially in determining LLL and LLLG fault types that were difficult to classify. From Table 2, it is evident that the addition of the new features significantly improved the performance of the models in all six fault types. The best results were obtained by CatBoost model with classification accuracy of 99.97%, followed by XGBoost, random forest, decision tree, and gradient boosting models with respective accuracies of 99.97%, 99.96%, 99.93% and 99.91%. Neural networks saw an increase in their accuracy from 84.80% to 92.48%, supporting the idea that feature engineering is advantageous for neural networks as well, although not as much as it was for DT. The accuracies attained by the other four classifiers after being subjected to feature engineering were 85.84%, 83.60%, 82.43% and 40.68%, respectively.

3.1.3 | Cross-Validation Before Hyperparameter Tuning

Before hyperparameters tuning, 10-fold cross-validation was carried out among the training data to serve as a generalization benchmark for all ten classifiers. All cross-validation figures stated below denote the mean accuracy obtained through all ten folds. In 10-fold cross-validation, the training data is partitioned into ten equal sets, and the model is trained ten times to ensure that each data subset is used to validate the model only once. As seen from Figure 14, the best mean cross-validation accuracy value is 90.26%, obtained using CatBoost, whereas the accuracy of the other four classifiers—random forest, gradient boosting, decision tree, and XGBoost—was 90.17%, 89.63%, 88.32% and 87.49% respectively. KNN reached a mean cross-validation accuracy score of 86.82%, and neural networks, naive Bayes, SVM and AdaBoost scored an accuracy of 84.76%, 81.74%, 81.60% and 75.79%, respectively. It should be noted that the cross-validation accuracies obtained before hyperparameter tuning were lower than the test accuracies, achieved through feature engineering

TABLE 2 | Performance comparison of models before and after feature engineering.

Model	Precision (%)	Recall (%)	F1-score (%)	Accuracy (%)
Before feature engineering				
Random forest	93.9716	93.2000	93.2391	93.2000
Decision tree	92.7947	92.4500	92.4406	92.4500
CatBoost	87.4790	86.5750	86.5503	86.5750
XGBoost	87.3880	86.4000	86.3367	86.4000
KNN	86.3127	85.1167	84.7521	85.1167
Gradient boosting	85.4589	84.8500	84.9022	84.8500
Neural networks	85.7643	84.8000	84.6109	84.8000
SVM	83.0946	81.1833	80.0398	81.1833
Naive Bayes	56.8524	58.3250	56.3925	58.3250
AdaBoost	39.9221	48.5667	43.2547	48.5667
After feature engineering				
CatBoost	99.9750	99.9750	99.9750	99.9750
XGBoost	99.9667	99.9667	99.9667	99.9667
Random forest	99.9583	99.9583	99.9583	99.9583
Decision tree	99.9333	99.9333	99.9333	99.9333
Gradient boosting	99.9083	99.9083	99.9083	99.9083
Neural networks	93.2008	92.4833	92.5576	92.4833
KNN	87.4238	85.8417	85.4204	85.8417
SVM	86.7640	83.6000	81.8123	83.6000
Naive Bayes	83.7795	82.4250	81.9950	82.4250
AdaBoost	60.1339	40.6833	31.1593	40.6833

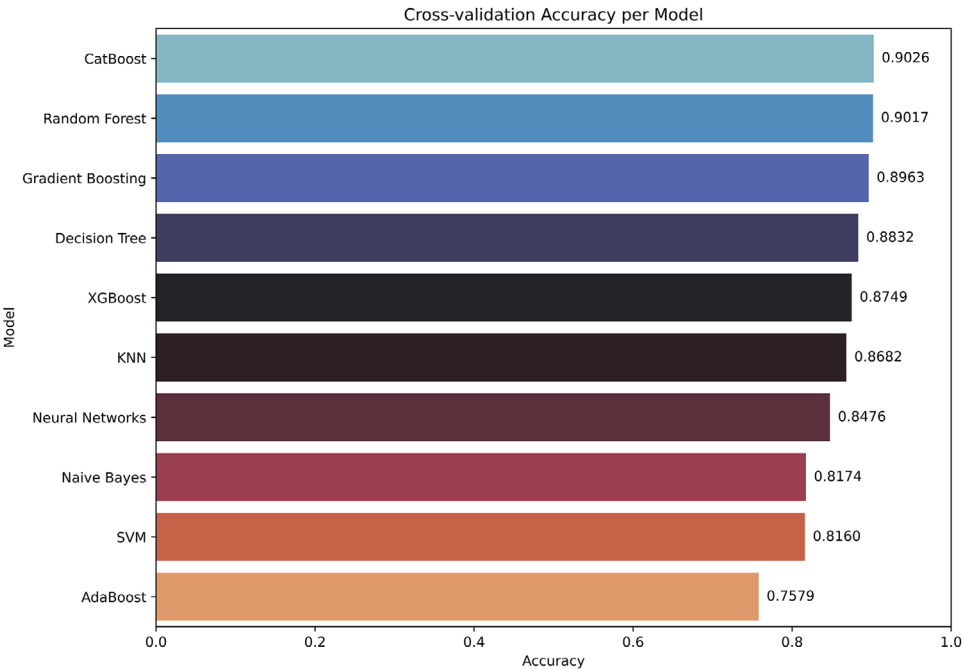


FIGURE 14 | Cross validation report based on accuracy.

after the tuning, due to two reasons: cross-validation was done before hyperparameter optimization; and in each cross-validation fold, the model was trained using about 90% of the training data or 72% of the entire dataset. The outcomes from the cross-validation prior to the tuning process were clear in demonstrating that both CatBoost and random forest exhibited the highest ability to generalize across all the classifiers considered, making them suitable choices for hyperparameter tuning.

3.1.4 | Hyperparameter Tuning

Following the 10-fold cross-validation results prior to tuning, random forest and CatBoost models were chosen for hyperparameter optimization using the randomized search cross-validation (RandomizedSearchCV) method. The reason for opting for the randomized search technique over the exhaustive grid search is its superior computational performance when searching for hyperparameters in large hyperparameter spaces, where rather than evaluating all available hyperparameter combinations exhaustively, a finite number of random hyperparameter combinations are sampled from the search space. The randomized search process utilized a 3-fold cross-validation internally, and the average validation accuracy metric was adopted as the criterion for selecting the best hyperparameters. For the Random Forest model, the hyperparameter search space was the number of estimators 50, 100, maximum tree depth none, 10, 20, minimum samples needed to split an internal node 2, 5, minimum samples needed at a leaf node 1, 2, and bootstrapping True. The search space for the CatBoost model consisted of boosting iterations 200, 500, 800, tree depth 4, 6, 8, 10, learning rate 0.01, 0.03, 0.05, 0.1, L2 regularization term 1, 3, 5, 7, 9 and border count 32, 64, 128. After determining the optimal hyperparameter values for each model via this procedure, they were employed to retrain the respective models on the entire training dataset.

3.1.5 | Cross-Validation After Hyperparameter Tuning

After tuning of hyperparameters, the next step was 3-fold cross-validation for the optimized versions of the random forest and CatBoost algorithms to evaluate how well the tuning has improved the ability of those classifiers to generalize their results. The application of 3-fold cross-validation in this case was chosen in accordance with the number of folds used in the randomized search process. All values are means calculated from all 3 folds.

From the classification report of the optimized random forest model obtained via cross-validation as shown in Table 3, it can be observed that the LG, LLL and no fault classes were able to attain perfect precision, recall and F1-score values of 1.00 on all three folds. The LL class scored a precision of 0.99 and F1-score of 0.99 whereas the LLG and LLLG classes scored 0.98 and 0.99 recall and F1-scores respectively, indicating slight confusion between asymmetrical and symmetrical fault signatures. In terms of cross-validation accuracy, macro average, and weighted average, the scores obtained were 1.00 each for all 60,000 samples, which shows a significant enhancement compared to the 90.17% accuracy attained by the pre-tuned model via 10-fold cross-validation. From the confusion matrix of the optimized random

TABLE 3 | Cross-validation classification report for random forest.

Class	Precision	Recall	F1-score	Support
LG	1.00	1.00	1.00	10,000
LL	0.99	1.00	0.99	10,000
LLG	1.00	1.00	1.00	10,000
LLL	1.00	1.00	1.00	10,000
LLLG	1.00	0.98	0.99	10,000
No fault	1.00	1.00	1.00	10,000
Accuracy		1.00		60,000
Macro Avg	1.00	1.00	1.00	60,000
Weighted Avg	1.00	1.00	1.00	60,000

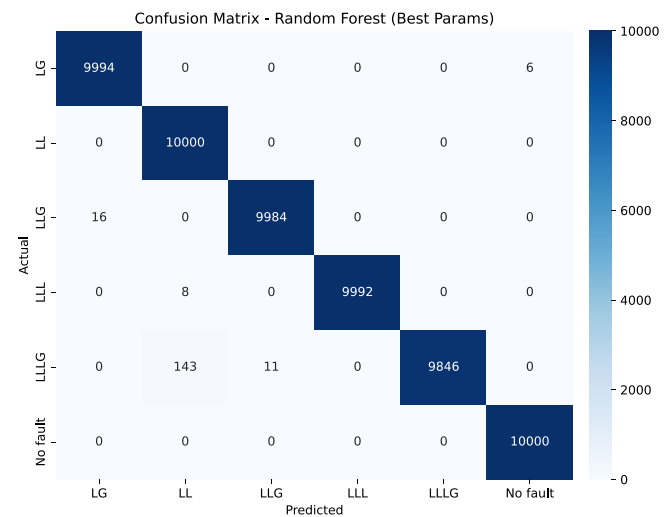


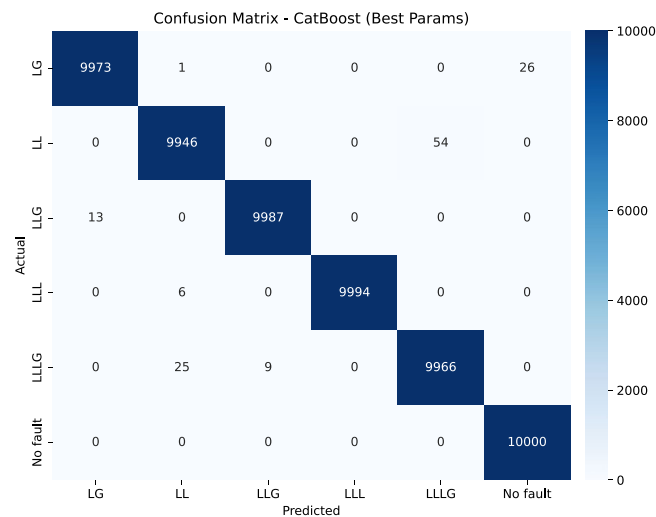
FIGURE 15 | Confusion matrix for random forest.

forest model presented in Figure 15, one can see that the LG class was able to obtain 9994 accurate classifications from a total of 10,000 samples, while the rest of the 6 samples were wrongly categorized as no fault. The LL fault class was perfectly classified with 10,000 correct predictions and no errors. The LLG fault class had 9984 correctly classified predictions, while there were 16 misclassified cases which had been classified as LG, implying small similarities between voltage and current values for the single line-to-ground and double line-to-ground faults. The LLLG class had 9848 predictions made correctly, out of which 143 were wrongly classified as LL fault, while 11 were classified as LLG, indicating the maximum number of misclassification among all fault types due to similarities in three-phase currents for three-phase fault and three-phase-to-ground fault conditions.

For the optimized CatBoost model, the cross-validation classification report presented in Table 4 demonstrates perfect 3-fold cross-validation performance across all six fault type classes, with precision, recall, and F1-score values of 1.00 for every individual class including LG, LL, LLG, LLL, LLLG and no fault. The overall cross-validation accuracy, macro average, and weighted average all achieved 1.00 across 60,000 total samples, improving substantially from the pre-tuning 10-fold cross-validation accuracy of 90.26 model illustrated in Figure 16 confirms this result, with the

TABLE 4 | Cross-validation classification report for Catboost.

Class	Precision	Recall	F1-score	Support
LG	1.00	1.00	1.00	10,000
LL	1.00	1.00	1.00	10,000
LLG	1.00	1.00	1.00	10,000
LLL	1.00	1.00	1.00	10,000
LLLG	0.99	1.00	1.00	10,000
No fault	1.00	1.00	1.00	10,000
Accuracy		1.00		60,000
Macro avg	1.00	1.00	1.00	60,000
Weighted avg	1.00	1.00	1.00	60,000

**FIGURE 16** | Confusion matrix for CatBoost.

most frequent misclassifications occurring between LL and LLG fault types, which exhibit overlapping electrical signatures under certain fault impedance conditions. The no fault class achieved perfect classification with 10,000 correct predictions across all folds, demonstrating the model's reliable identification of healthy system conditions.

3.2 | Location Prediction

The models developed successfully predicted the fault location that occurred in the transmission line with minimal error. The predicted fault distances or the predicted fault location closely match the actual values with high precision and accuracy, with most predictions falling within a $\pm 1\%$ tolerance range. This high precision demonstrates the model's suitability for real-time fault monitoring and protection systems. Additionally, the model's performance remained consistent across various fault types and locations, confirming its robustness. Overall, the results validate the effectiveness of ML and DL in precise fault localization. In terms of classical ML models, random forest regressor was the best regressor with the highest R^2 and the lowest MAE and RMSE. While in the case of DL models, one dimensional CNN (1D CNN) was the best model, observing R^2 , MAE and RMSE.

TABLE 5 | Performance comparison of machine learning and deep learning models.

Model	MAE	RMSE	R^2
Classic machine learning			
XGB regressor	0.1287	0.2314	0.9996
Linear regression	2.8380	3.8981	0.8845
Random forest regressor	0.0080	0.0729	1.0000
Decision tree regressor	0.0127	0.1549	0.9998
Gradient boosting regressor	0.5652	1.1577	0.9898
Deep learning and CNN models			
Artificial neural network (ANN)	1.1899	1.4176	0.9847
Long short-term memory (LSTM)	0.6601	0.9857	0.9926
1D convolutional neural network (1D CNN)	0.4978	0.7252	0.9969
Temporal convolutional network (TCN)	0.5249	0.8417	0.9946

TABLE 6 | Test accuracy (%) at different signal-to-noise ratio (SNR) levels.

Model	0 dB	10 dB	20 dB	30 dB	40 dB	50 dB
CatBoost	99.98	77.18	84.58	87.04	88.53	90.53
XGBoost	99.97	76.89	84.83	86.74	88.37	90.36
Random forest	99.96	76.90	84.46	86.78	88.21	89.94
Decision tree	99.93	75.89	84.54	87.17	88.82	90.58
Gradient boosting	99.91	75.89	84.54	87.17	88.82	90.58
Neural networks	90.37	70.09	82.03	84.81	85.01	87.93
KNN	85.84	68.68	81.18	84.21	84.78	85.24
SVM	83.60	74.87	82.29	85.03	86.03	88.18
Naive Bayes	82.43	69.58	74.95	76.62	77.86	79.40
AdaBoost	40.68	51.43	51.30	77.11	68.67	59.33

Table 5 compares all the models used for LP, which also justifies the result.

3.3 | Noise

This part compares the effects of noise on model performance at different SNRs. The assessment is broken down into two sections, namely classification and regression, which bring to light the response of various learning paradigms to noise degradation. This section determines the robustness patterns, critical degradation points, and recovery behavior of the models in realistic noisy conditions by comparing performance in low SNR (high noise) and high SNR (low noise).

3.3.1 | Result After Addition of Noise in FTC

Table 6 shows the accuracy of different ML models at different SNR levels. At different noise levels, decision tree and

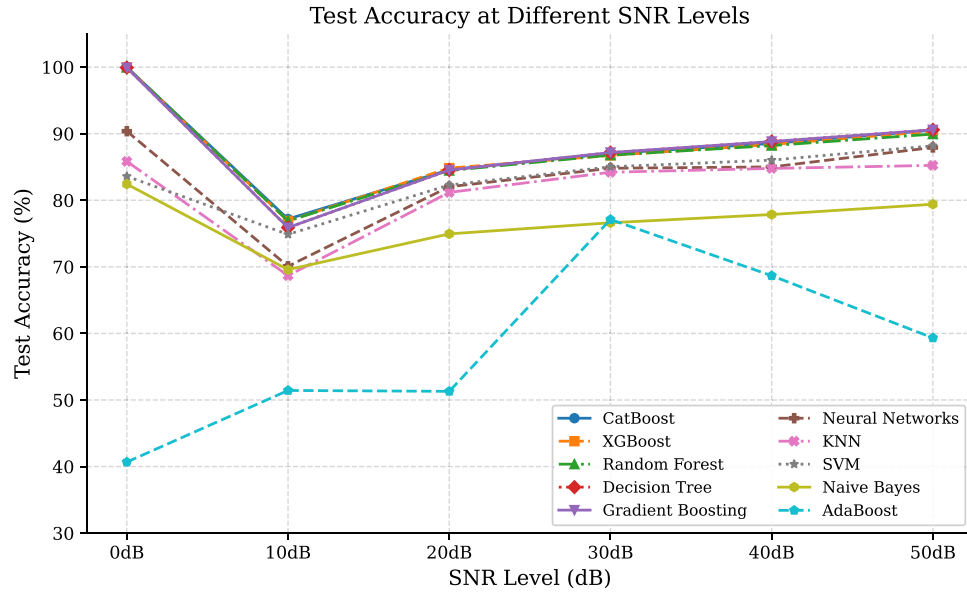


FIGURE 17 | Test accuracy (%) at different SNR levels.

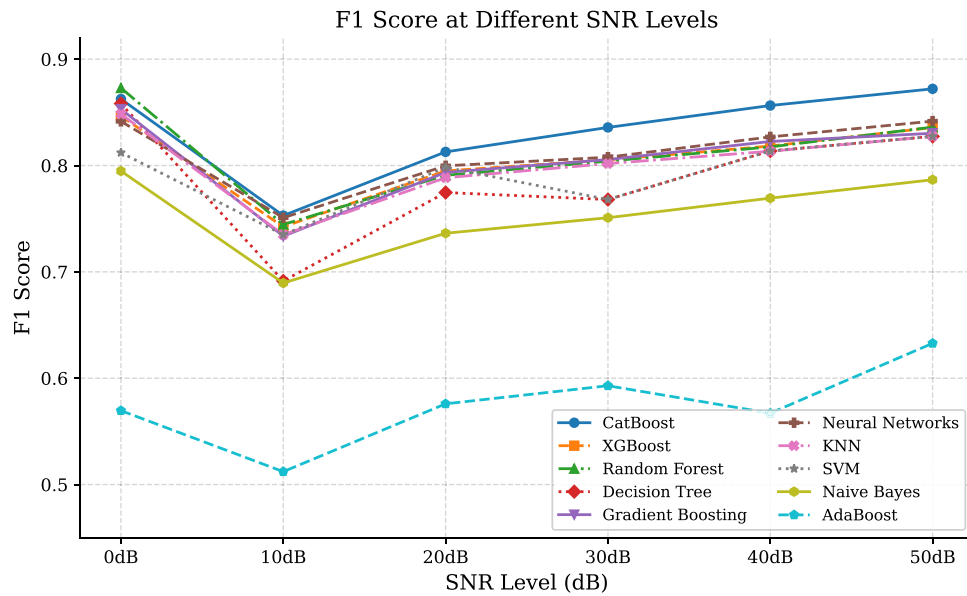


FIGURE 18 | Cross-validation accuracy at different SNR levels.

gradient boosting performed better than other models with test accuracy of 90.58% in a 50 dB noise level. Also, the comparison of test accuracies of different ML models is illustrated by Figure 17.

To evaluate the models more accurately cross-validation was performed as shown in Table 6, the which showed CatBoost as the more robust model with a CV accuracy of 87.21% and F1-score of 0.88 at 50 dB noise level. CatBoost at 10 dB also had a slight difference between test accuracy and cross-validation score, which means that it generalizes well despite the presence of noise. Therefore, CatBoost offers the optimal accuracy, strength and generalization at all SNR levels which can be visualized from Table 6 and Figure 18.

In general, classification analysis indicates that ensemble learning techniques are the most appropriate to use in the real world where there is some noise. Their multi-model architecture enables them to be tolerant of corrupted features and to perform more reliably under varying noise conditions.

3.3.2 | ROC Curve Analysis for FTC

For further evaluation, receiver operating characteristic (ROC) curves were also calculated for the two models that performed best for noise at all SNR levels for both random forest and CatBoost. AUC values verify good class separation in both models even in the presence of high noise. Figures 19 and 20 show

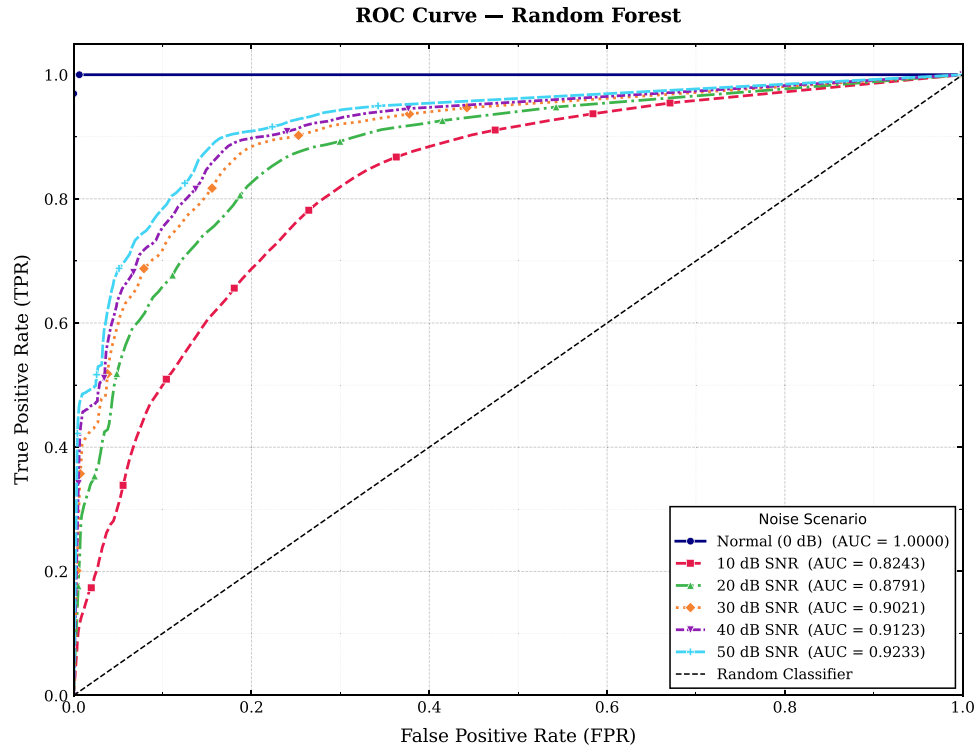


FIGURE 19 | ROC curves of the random forest classifier at different SNR levels.

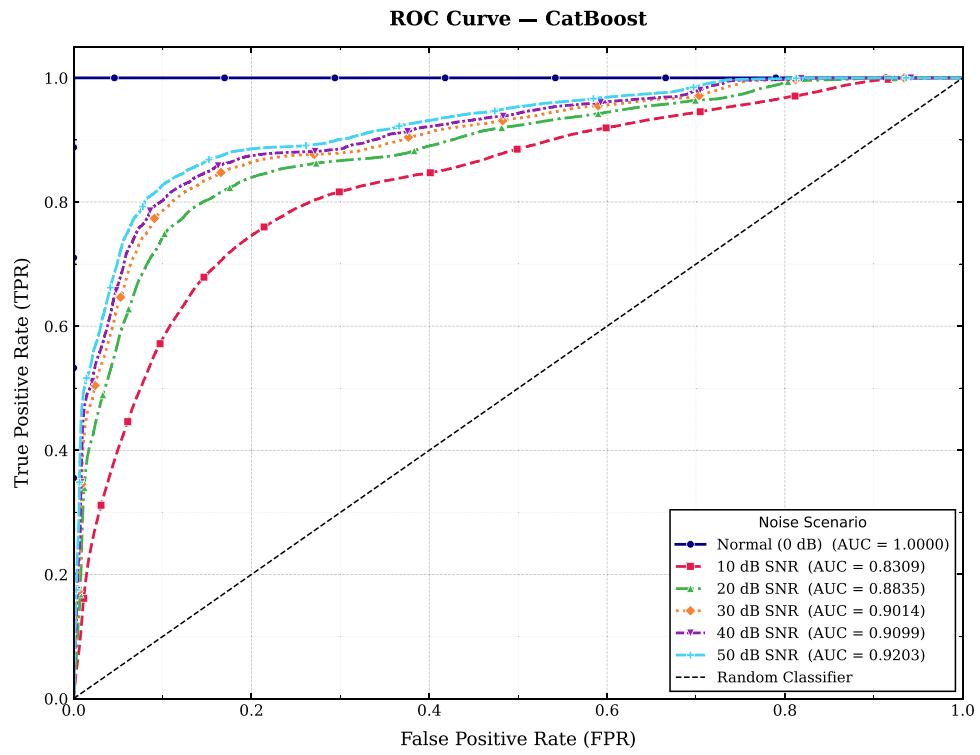


FIGURE 20 | ROC curves of the CatBoost classifier at different SNR levels.

TABLE 7 | Cross-validation accuracy at different signal-to-noise ratio (SNR) levels.

Model	0 dB	10 dB	20 dB	30 dB	40 dB	50 dB
Random forest	0.8729	0.7445	0.7908	0.8041	0.8175	0.8361
CatBoost	0.8625	0.7529	0.8128	0.8358	0.8564	0.8721
Decision tree	0.8582	0.6914	0.7746	0.7680	0.8137	0.8274
Gradient boosting	0.8530	0.7336	0.7935	0.8056	0.8226	0.8303
KNN	0.8484	0.7349	0.7885	0.8018	0.8134	0.8274
XGBoost	0.8465	0.7424	0.7949	0.8053	0.8183	0.8357
Neural networks	0.8415	0.7509	0.7997	0.8078	0.8269	0.8417
SVM	0.8121	0.7349	0.7983	0.7680	0.8137	0.8274
Naive Bayes	0.7948	0.6895	0.7363	0.7509	0.7693	0.7866
AdaBoost	0.5695	0.5121	0.5759	0.5929	0.5671	0.6328

the ROC curve for random forest and CatBoost, respectively. CatBoost consistently outperforms all other algorithms in terms of AUC on all noise levels, further strengthening its robustness and generalization ability.

When the signal is of good quality (0 dB noise-free), the random forest classifier demonstrated a perfect AUC value of 1.0000 and proved its full discriminative power, as shown in Figure 19. When the noise level is 10 dB SNR (the worst noise level in this study), however, the AUC value turns to 0.8243, which shows that the voltage and current signals features used in this model is greatly affected by high noise. The AUC recovers consistently and monotonically from 20 dB up to 50 dB SNR, recovering to 0.8791, 0.9021, 0.9123 and 0.9233 respectively. The progressive improvement shows that the Random Forest is noise-sensitive and that the classification accuracy of the image is directly proportional to the quality of the image signal. The system shows a good performance in moderate to high SNR however the low AUC values in the range 10 dB indicates a limited robustness in the extreme noise scenario.

The ROC curves of CatBoost classifier are shown in Figure 20 under the same noise conditions. At 0 dB, CatBoost had an AUC of 1.0000, which is the same as Random Forest. CatBoost performed better under noisy conditions with an AUC at SNR 10 dB of 0.8309, significantly better than the AUC of 0.8243 achieved by random forest under the same conditions. CatBoost exhibited superior performance over random forest at all other SNR with AUC values of 0.8835, 0.9014, 0.9099 and 0.9203 in SNR 20 dB, SNR 30 dB, SNR 40 dB and SNR 50 dB respectively. This regular margin gain for all noise levels verifies that CatBoost generalizes better in the presence of signal degradation, which is believed to be due to its ordered boosting mechanism and regularization built into the model to make it less sensitive to corrupted feature values. The results from cross-validation in Table 7 suggest that in all SNR levels CatBoost outperforms other models in generalization. Thus, CatBoost is revealed to be the most powerful and stable classifier for fault type classification in real-world noisy power system environment.

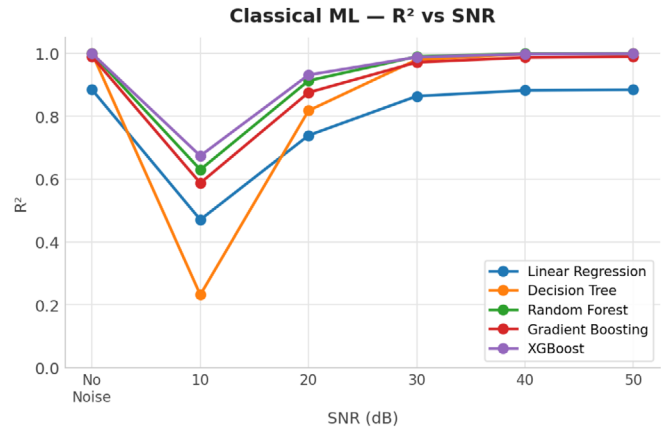


FIGURE 21 | Performance of classical ML models across SNR levels.

3.3.3 | Result After Addition of Noise in LP

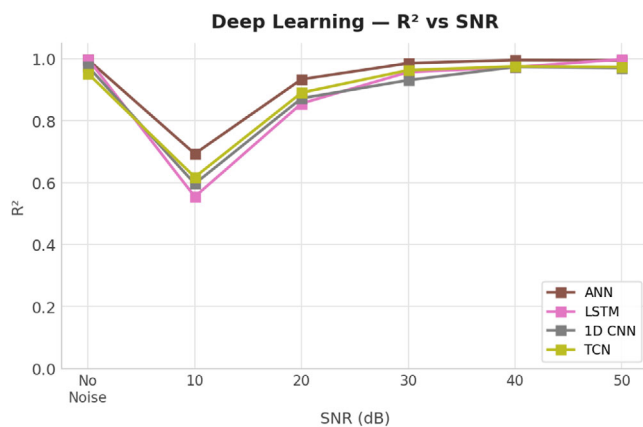
Regression analysis makes an interesting discovery: the great clean-data performance does not imply the high strength in noisy conditions. A number of models performed almost perfectly on noise-free data. Random forest regressor ($R^2 = 1.0000$ and $MAE = 0.008$) and decision tree ($R^2 = 0.9998$) and XGB regressor ($R^2 = 0.9996$) also did very well. Deep learning models like CNN, LSTM, TCN also showed very high accuracy. These findings affirm that sophisticated models are able to fit clean datasets.

Nonetheless, when noise was introduced at SNR = 10 dB, the effect of performance degradation was significant. The worst collapse was observed with decision tree regressor which collapsed to $R^2 = 0.2319$ which was less than the original $R^2 = 0.9998$. This means that the trees based on precise thresholds are very vulnerable to noisy changes in features. Random forest also dropped drastically to $R^2 = 0.6299$, yet it was still better than the decision tree since the average of the ensembles decreases the effect of the volatile splits. XGB Regressor produced $R^2 = 0.6739$ and was more robust, as it optimized and regularized. From Table 8, it is clear that, in classical ML models, Random Forest regressor was the most sturdy model with R^2 of 0.9999 whereas, LSTM was more strong to noise among DL models with R^2 of 0.9981 in 50 dB noise level. R^2 of different models at different noise levels can be envisioned from Figures 21 and 22.

It should be acknowledged that noise is not the only real-world factor that can influence model performance. The present study deliberately varied two such factors during data generation: fault resistance (0.001 Ω to 50 Ω , covering both solid and high-impedance faults) and fault inception angle (0° to 315° in steps of 45°), so that the trained models are inherently exposed to this variability rather than being evaluated against it separately. Beyond these, several additional factors were not considered in this study and remain open for future investigation: load variation (the present study uses a fixed three-phase load, and changes in loading conditions may alter the pre-fault voltage profile and apparent fault signatures), network topology changes (the model was trained on a single-line, two-bus configuration and has not been evaluated under altered topologies such as line outages or ring-bus operation), and instrument transformer (CT/PT) measurement errors, which introduce systematic biases

TABLE 8 | R^2 score comparison of machine learning and deep learning models under various noise conditions.

Model	No noise	10 dB	20 dB	30 dB	40 dB	50 dB
Classic machine learning						
XGB regressor	0.9996	0.6739	0.9316	0.9884	0.9982	0.9992
Linear regressor	0.8845	0.4705	0.7389	0.8640	0.8824	0.8843
Random forest regressor	1.0000	0.6299	0.9130	0.9902	0.9993	0.9999
Decision tree regressor	0.9998	0.2319	0.8179	0.9800	0.9984	0.9996
Gradient boosting regressor	0.9898	0.5872	0.8755	0.9718	0.9871	0.9899
Deep learning models						
ANN	0.9971	0.6933	0.9339	0.9858	0.9958	0.9950
LSTM	0.9989	0.5547	0.8558	0.9569	0.9731	0.9981
1D CNN	0.9749	0.5965	0.8725	0.9309	0.9738	0.9708
TCN	0.9520	0.6180	0.8906	0.9634	0.9758	0.9739

**FIGURE 22** | Performance of deep learning models across SNR levels.

distinct from the random Gaussian noise modelled here. These factors are discussed further in the limitations section.

4 | Limitations of the Study

The results of the current research prove the high performance and potential of ML and DL algorithms for automated fault detection and localization in power transmission networks as well as provide a feasible and scalable alternative to traditional protection systems that involve mathematical models of faults. Using the synthetically generated dataset provided by the MATLAB/Simulink simulation models provides an advantage of a controlled and well-balanced experimental design; yet, this is one of the major limitations of the current research, as the developed models have been validated solely in simulation and not yet in field measurements where noise and uncertainty may affect the results. While fault resistance and inception angle were varied during data generation, other practical factors such as load variation, network topology changes and instrument transformer (CT/PT) measurement errors were not considered and may further affect model performance in real-world deployment. Additionally, computational requirements for applying

ensemble algorithms, such as random forest and CatBoost, in real time in digital protective relays' hardware have not been explored in the current research and should be considered in future works. The areas of further research include real-world data collection, validation, and testing of the developed ML/DL models on faults occurring in power transmission networks, studying model robustness under changing impedance, fault angle, loads, network topology and measurement errors, applying the approach in distribution and microgrids, as well as creating lighter architecture for embeddability. Also, exploring the addition of the negative sequence components to the feature set can be proposed in future research to enhance asymmetrical fault detection and localization performance.

5 | Computational Environment

All ML and DL models were implemented in Python 3.10 using Google Colaboratory (Google Colab) as the development and execution environment, which provides cloud-based access to GPU-accelerated computing resources. Classical ML algorithms were developed using scikit-learn (v1.3), while XGBoost (v2.0) and CatBoost (v1.2) were implemented using their respective dedicated Python libraries, installed via pip within the Colab runtime. Deep learning models including ANN, LSTM, 1D CNN and TCN were built and trained using TensorFlow/Keras (v2.13). Data preprocessing utilized pandas and NumPy, and all visualizations were produced using Matplotlib and Seaborn. The MATLAB/Simulink model used for synthetic fault data generation was executed in MATLAB R2023a on a local machine. All model training and evaluation experiments were conducted on Google Colab's cloud runtime, utilizing an NVIDIA T4 GPU (15 GB VRAM) provided by the platform to accelerate deep learning model training. Classical ML models including decision tree and KNN trained in under 1 min, while CatBoost and random forest with RandomizedSearchCV hyperparameter tuning required approximately 20–35 min. Deep learning models (ANN, LSTM, 1D CNN, TCN) were each trained for up to 100 epochs with a batch size of 16 and a validation split of 20%, using the Adam optimizer with a learning rate of 0.001 and mean squared error (MSE) as the loss function for regression tasks.

6 | Conclusion

Fault detection accuracy and speed are paramount to ensuring stable operation, mitigating any potential damage to equipment, and minimizing the downtime of the electrical network. Conventional fault detection techniques such as impedance analysis can be prone to certain limitations such as sensitivity to varying system operating conditions, fault impedances and network topology changes. To overcome these challenges, this paper developed an extensive ML/DL approach for simultaneously estimating the type of fault FTC and FLE. An artificial fault data generation process was conducted through MATLAB/Simulink, with the consideration of six types of faults, including single line to ground LG, line to line LL, LLG, LLL, LLLG and no fault. Ten classifiers and six regression algorithms were assessed for FTC and FLE, respectively.

The proposed framework for fault type classification showed exceptional performance after the feature engineering that considered the inclusion of V_0 and I_0 as additional features in addition to the three phases' voltage and currents. For the baseline test set performance before feature engineering, the best model was the random forest classifier, whose test set accuracy was 93.20%. After feature engineering, the most accurate classifier was CatBoost with 99.97% accuracy, followed by XGBoost with 99.97% accuracy, random forest with 99.96% accuracy, decision tree with 99.93% accuracy and gradient boosting with 99.91% accuracy. The presence of the zero-sequence features played an important role in distinguishing the asymmetrical fault types LG and LLG from the three symmetrical phases LLL and LLLG which had identical three-phases electric features before. After tuning the hyperparameters using RandomizedSearchCV, the generalization capabilities of CatBoost and random forest have been improved with the post-tuning 3-fold cross-validation that resulted in perfectly classified six fault type classes having a score of 1.00 in accuracy, precision, recall and F1-score. Pre-tuning 10-fold cross-validation scores were 90.26% accuracy for CatBoost and 90.17% for random forest model.

As we move from fault type classification to fault location estimation, the models showed very promising prediction performance in both classic ML algorithms and deep learning methods. In terms of fault location performance in classical ML regressors, the model that performed best is the random forest, with a coefficient of determination of $R^2 = 1.0000$, a mean absolute error of $MAE = 0.0380$, and a root mean squared error of $RMSE = 0.0729$. This means very accurate distance predictions were made for all fault locations using the model. It should be noted that a perfect performance of $R^2 = 1.0000$ was obtained from a testing dataset created within MATLAB/Simulink, where there could be slight differences in real-life applications due to different factors. Decision tree showed the best MAE score of 0.0127, whereas gradient boosting and XGBoost also proved to be highly efficient methods with R^2 values of 0.9998 and 0.9996 respectively. The worst performing method was linear regression, with an R^2 value of 0.8800, an MAE score of 2.3830, and RMSE of 3.8381. This indicated that linear regression could not predict nonlinear relationships between phasor characteristics and fault distance. From the perspective of deep learning models, the best performing model was TCN with an R^2 score of 0.9946, 1D CNN with 0.9906, LSTM with 0.9857, and ANN with 0.9847.

Overall, classical machine learning algorithms outperformed deep learning methods in this tabular phasor dataset case study, which implies that feature exploitation capabilities of random forest and other similar models suit this problem more than deep learning methods.

The models can behave ideal in synthetic data, so we moved on to add noise to the data so that the data becomes more closer to the real time data during the fault conditions. After the addition of noise, the best model was CatBoost with CV accuracy of 87.21% and F1 score of 0.88 at the noise level of 50 dB. Thus, the result supports that the model was robust and can predict the fault type in real world scenario. Moreover, in the case of LP, Random forest regressor was the best model among the classical ML approaches before and after the addition of noise with the R^2 of 0.9999 at 50 dB noise level. However, in the case of DL techniques, LSTM was the strongest model with R^2 of 0.9981.

Author Contributions

Subash Khanal: conceptualization, formal analysis, investigation, methodology, software, validation, visualization. **Subarna Khadka:** conceptualization, formal analysis, software, writing – original draft. **Bipun Man Pati:** formal analysis, supervision, validation, writing – review and editing. **Sudesh Parajuli:** software, validation, writing – original draft, writing – review and editing.

Funding

The authors have nothing to report.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study were generated using a MATLAB Simulink simulation model. The dataset used for fault classification and location prediction in this study is available from the corresponding author upon reasonable request.

References

1. M. O. F. Goni, M. Nahiduzzaman, M. S. Anower, et al., "Fast and Accurate Fault Detection and Classification in Transmission Lines Using Extreme Learning Machine," *e-Prime - Advances in Electrical Engineering, Electronics and Energy* 3, no. 1 (January 2023): 100107, <https://doi.org/10.1016/j.prime.2023.100107>.
2. A. Rahmati and R. Adhemi, "A Fault Detection and Classification Technique Based on Sequential Components," *IEEE Transactions on Industry Applications* 50, no. 6 (2014): 4202–4212, <https://doi.org/10.1109/TIA.2014.2313652>.
3. H. Teimourzadeh, M. R. Iravani, and A. Yazdani, "High Impedance Single Phase Fault Diagnosis in Transmission Lines via Convolutional Neural Networks and Deep Reinforcement Learning," *IEEE Transactions on Smart Grid* 14, no. 1 (2023): 456–467, <https://doi.org/10.1109/TSG.2022.3226789>.
4. S. Bera, P. Roy, and A. Sinha, "Autoregressive Coefficient-Based Intelligent Protection for Transmission Line Fault Detection Using Deep Learning," *Expert Systems with Applications* 218 (2023): 119471, <https://doi.org/10.1016/j.eswa.2023.119471>.
5. M. Wadi and W. Elmasry, "Anomaly-Based Fault Detection Using One-Class SVM and PCA," *International Journal of Electrical Power & Energy Systems* 143 (2023): 108517, <https://doi.org/10.1016/j.ijepes.2022.108517>.

6. Y. D. Mamuya, Y.-D. Lee, J.-W. Shen, M. Shafiullah, and C.-C. Kuo, "Application of Machine Learning for Fault Classification and Location in a Radial Distribution Grid," *Applied Sciences* 10, no. 14 (2020): 4965, <https://doi.org/10.3390/app10144965>.
7. T. Tanyildizi, H. Kaya, and M. Yildirim, "CNN Based Classification of Ground Short Circuit Faults in Transmission Lines," *Energies* 17, no. 3 (2024): 872, <https://doi.org/10.3390/en17030872>.
8. D. Novosel, B. Kasztenny, E. O. Schweitzer III, and M. J. Hughes, "Method and System for Determining Location of a Fault on a Power Transmission Line," Patent U.S. Patent 6879917B2, (April 2005), issued April 12, 2005.
9. U. Umathe, P. Daigavane, and M. Daigavane, "An Efficient Technique to Improve Fault Categorization in Transmission Lines," *Engineering, Technology & Applied Science Research* 15, no. 2 (2025): 21425–21430.
10. Y. Li, J. Wang, and Z. Chen, "Optimized Machine Learning Algorithms for Fault Detection, Classification & Localization in Transmission Lines," *International Journal of Computational Intelligence Systems* 17, no. 1 (2024): 123–135, <https://doi.org/10.1007/s44196-024-00434-7>.
11. M. Bon and X. Dai, "Hybrid Machine Learning Methods for Fault Identification and Precise Location in High Voltage Transmission Lines," *Journal of Electrical Engineering & Technology* 19, no. 4 (2022): 2218–2230, <https://doi.org/10.1007/s42835-022-01234-5>.
12. T. N. Hung, "Methods for Fault Location in High Voltage Power Transmission Lines: A Comparative Analysis," *International Journal of Renewable Energy Development* 11, no. 4 (2022): 1134–1141, <https://doi.org/10.14710/ijred.2022.46501>.
13. R. Fan, T. Yin, R. Huang, J. Lian, and S. Wang, "Ensemble Machine Learning for Robust Fault Detection and Location in Power Transmission Lines," *Scientific Reports* 15 (2025): 98, <https://doi.org/10.1038/s41598-025-86554-2>.
14. S. B. Akter, T. S. Pias, S. R. Deeba, J. Hossain, and H. A. Rahman, "Ensemble Learning Based Transmission Line Fault Classification Using Phasor Measurement Unit (PMU) Data With Explainable AI (XAI)," *PLOS ONE* 19, no. 2 (2024): e0295144, <https://doi.org/10.1371/journal.pone.0295144>.
15. P. P. Wasnik, N. J. Phadkule, and K. D. Thakur, "Fault Detection and Classification Based on Semi-Supervised Machine Learning Using KNN," in *Proceedings of the International Conference on Innovative Trends and Advances in Engineering and Technology (ICITAET)* (IEEE, 2019), 79–83.
16. W. Zhang, Z. Li, and Q. Huang, "Deep Learning Based Detection of Transmission Line Insulator Defects Using UAV Imagery," *Energies* 16, no. 14 (2023): 5560, <https://doi.org/10.3390/en16145560>.
17. S. Chakrabarti and A. Swetapadma, "A Review on Various Artificial Intelligence Techniques Used for Transmission Line Fault Location," in *Proceedings of the International Conference on Inventive Computation Technologies (ICICT)* (IEEE, 2018), 105–110.
18. S. Turanlı and D. Yakut, "Classification of Faults in Transmission Lines Using Deep Learning and Real/Synthetic Data," *Applied Sciences* 14, no. 20 (2024): 9590, <https://doi.org/10.3390/app14209590>.
19. T. Tusher, M. Rahman, and A. Islam, "A Comparative Analysis of Different Transmission Line Fault Detectors and Classifiers During Normal Conditions and Cyber Attacks," *The Journal of Engineering* 2024, no. 18 (2024): 1–12, <https://doi.org/10.1049/tje.2024.12412>.
20. F. M. Shakiba, S. M. Azizi, M. Zhou, and A. Abusorrah, "Application of Machine Learning Methods in Fault Detection and Classification of Power Transmission Lines: A Survey," *Artificial Intelligence Review* 56, no. 7 (2023): 5799–5836, <https://doi.org/10.1007/s10462-022-10296-0>.
21. P. Nayak, S. R. Das, R. K. Mallick, et al., "2D-Convolutional Neural Network Based Fault Detection and Classification of Transmission Lines Using Scalogram Images," *Heliyon* 10, no. 4 (2024): e14978, <https://doi.org/10.1016/j.heliyon.2024.e14978>.
22. S. Wang, H. Chen, and L. Zhang, "Time Frequency Embedded Deep Learning for Incipient Fault Detection in Transmission Lines," *IEEE Transactions on Power Delivery* 38, no. 3 (2023): 2045–2055, <https://doi.org/10.1109/TPWRD.2023.3234567>.
23. N. Q. Minh, N. T. Khiem, and V. H. Giang, "Fault Classification and Localization in Power Transmission line Based on Machine Learning and Combined CNN–LSTM Models," *Energy Reports* 12 (2024): 5610–5622, <https://doi.org/10.1016/j.egy.2024.10.123>.
24. M. Silva, B. A. Souza, and N. S. D. Brito, "Fault Detection and Classification in Transmission Lines Based on Wavelet Transform and ANN," *IEEE Transactions on Power Delivery* 21, no. 4 (2006): 2059–2063, <https://doi.org/10.1109/TPWRD.2006.876659>.
25. C. Chowdary, "Machine Learning Algorithms for Transmission Line Fault Detection: A Comparative Study," *International Journal of Intelligent Systems and Applications in Engineering* 12, no. 23 (2024): 2684–2692.
26. S. Khan, M. U. Rehman, and A. N. Qureshi, "Image Based Detection of Insulator Defects via Deep Learning in Transmission Line Inspection," *Sensors* 23, no. 5 (2024): 2124, <https://doi.org/10.3390/s23052124>.
27. R. Fan, T. Yin, R. Huang, J. Lian, and S. Wang, "Transmission Line Fault Location Using Deep Learning Techniques," in *Proceedings of the 51st North American Power Symposium (NAPS)* (IEEE, 2019), 1–6, <https://doi.org/10.1109/NAPS.2019.8914857>.
28. Y. Liu, Y. Xing, D. Lu, X. Zou, and X. He, "A Physics-Informed Data Driven Fault Location Method for Transmission Lines Using Single Ended Measurements With Field Data Validation," *arXiv:2307.09740* (2023), <https://doi.org/10.48550/arXiv.2307.09740>.
29. P. V. Dhole, "Deep Neural Network Based Transmission Line Fault Location Algorithm," *International Journal of Intelligent Systems and Applications in Engineering* 12, no. 4 (2024): 1369–1378.
30. C. G. Arsoniadis and V. C. Nikolaidis, "Deep Learning Through LSTM Classification and Regression for Transmission Line Fault Detection, Diagnosis and Location in Large-Scale Multi-Machine Power Systems," *Renewable Energy* 198 (2022): 75–90, <https://doi.org/10.1016/j.renene.2022.08.020>.
31. Y. Lu, X. Zheng, H. Gao, X. Ding, and X. Liu, "A Novel Convolutional LSTM Network Based on Enhanced Feature Extraction for Transmission Line Fault Diagnosis," *Processes* 11, no. 10 (2023): 2955, <https://doi.org/10.3390/pr11102955>.
32. Y. Xing, J. Li, M. Zhou, and H. Wang, "Transmission Line Fault Classification Using Conformer Convolution-Augmented Transformer Model," *Applied Sciences* 14, no. 10 (2023): 4031, <https://doi.org/10.3390/app14104031>.
33. Z. Bai, X. Zhao, and L. Dong, "3D Deep Learning for Classifying Power System Faults Using 4D Sigmoid Images," *IEEE Transactions on Power Systems* 38, no. 4 (2023): 3456–3465, <https://doi.org/10.1109/TPWRS.2023.3245678>.
34. F. Gilanifar, M. R. Fattahi, and S. H. Hosseini, "Multi-Task Latent Structure Learning for Simultaneous Fault Classification and Location in Power Systems," *Electric Power Systems Research* 222 (2024): 108274, <https://doi.org/10.1016/j.epsr.2023.108274>.
35. S. A. Kharusi, H. E. Haffar, and M. Mesbah, "Adaptive Machine Learning Based Transmission Line Fault Detection and Classification Connected to Inverter Based Generators," *Energies* 16, no. 15 (2023): 5775, <https://doi.org/10.3390/en16155775>.
36. T. Tikariha, N. K. Mohapatra, and S. Panda, "CNN Based Fault Classification for IEEE 30 Bus System," *International Journal of Electrical Power & Energy Systems* 132 (2022): 107114, <https://doi.org/10.1016/j.ijepes.2021.107114>.
37. S. K. Chaudhary, S. Khanal, P. K. Chaudhary, B. Pancha, and J. K. Yadav, "Multi-Class Fault Type Classification Using Sequence Components and Wavelet Features with Comparative Study of Machine Learning Models and Neural Networks," in *2026 International Conference on ICT and Photonics (ICTP)*, vol. 1 (2026), 1–5, <https://doi.org/10.1109/ICTP67998.2026.11485076>.